

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

Oleh:

Hamka Arifani

NIM. 2410817110006

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile . Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Hamka Arifani
NIM : 2410817110006

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
SOAL 1	1
A. Source Code	1
B. Output Program	49
C. Pembahasan	52
SOAL 2	65
A. Pembahasan.....	65
TAUTAN GIT.....	66

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Logging Timber Soal 1 XML.....	49
Gambar 2. Screenshot Hasil Debugging Timber Soal 1 XML	50
Gambar 3. Screenshot Hasil Logging Timber Soal 1 Compose	50
Gambar 4. Screenshot Hasil Logging Timber Soal 1 Compose	51
Gambar 5. Screenshot Hasil Debugging Timber Soal 1 Compose	51

DAFTAR TABEL

Tabel 1. Source Code Soal 1 MainActivity.kt XML	1
Tabel 2. Source Code Soal 1 Film.kt XML	2
Tabel 3. Source Code Soal 1 FilmViewModel.kt XML	3
Tabel 4. Source Code Soal 1 FilmViewModelFactory.kt XML	4
Tabel 5. Source Code Soal 1 DetailViewModel.kt XML	4
Tabel 6. Source Code Soal 1 DetailViewModelFactory.kt XML	5
Tabel 7. Source Code Soal 1 LanguageViewModel.kt XML	5
Tabel 8. Source Code Soal 1 LanguageViewModelFactory.kt XML	6
Tabel 9. Source Code Soal 1 DataSource.kt XML	6
Tabel 10. Source Code Soal 1 CarouselAdapter XML	8
Tabel 11. Source Code Soal 1 MovieListAdapter XML.....	9
Tabel 12. Source Code Soal HomeFragment.kt XML	10
Tabel 13. Source Code Soal 1 DetailFragment XML	13
Tabel 14. Source Code Soal 1 LanguageFragment XML	14
Tabel 15. Source Code Soal 1 Activity_Main.XML.....	15
Tabel 16. Source Code Soal 1 Fragmenthome.XML.....	16
Tabel 17. Source Code Soal 1 Fragmentdetail.XML	19
Tabel 18. Source Code Soal 1 Fragmentlanguage.XML	22
Tabel 19. Source Code Soal 1 Header_back.XML	23
Tabel 20. Source Code Soal 1 Item_film.XML	24
Tabel 21. Source Code Soal 1 item_highlight.XML.....	28
Tabel 22. Source Code Soal 1 nav_graph.XML	28
Tabel 23. Source Code Soal 1 MainActivity.kt Compose	29
Tabel 24. Source Code Soal 1 Film.kt Compose	30
Tabel 25. Source Code Soal 1 DataSource.kt Compose	30
Tabel 26. Source Code Soal 1 FilmViewModel .kt Compose	32
Tabel 27. Source Code Soal 1 FilmViewModelFactory .kt Compose.....	32
Tabel 28. Source Code Soal 1 DetailViewModel .kt Compose.....	33
Tabel 29. Source Code Soal 1 DetailViewModelFactory .kt Compose.....	33
Tabel 30. Source Code Soal 1 LanguageViewModel .kt Compose.....	34
Tabel 31. Source Code Soal 1 LanguageViewModelFactory .kt Compose.....	34
Tabel 32. Source Code Soal 1 HeaderScreen.kt Compose	35
Tabel 33. Source Code Soal 1 HomeScreen.kt Compose	36
Tabel 34. Source Code Soal 1 DetailScreen.kt Compose	41
Tabel 35. Source Code Soal 1 LanguageScreen.kt Compose	45
Tabel 36. Source Code Soal 1 Nav_Graph.kt Compose	47

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item.
Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

XML

MainActivity.kt

Tabel 1. Source Code Soal 1 MainActivity.kt XML

1	package com.example.filafixxml
2	
3	import android.os.Bundle
4	import androidx.activity.enableEdgeToEdge
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.core.view.ViewCompat
7	import androidx.core.view.WindowInsetsCompat
8	import androidx.navigation.fragment.NavHostFragment
9	import androidx.navigation.ui.setupActionBarWithNavController
10	import com.example.filafixxml.databinding.ActivityMainBinding
11	
12	class MainActivity : AppCompatActivity() {
13	private val binding: ActivityMainBinding by lazy {

14	ActivityMainBinding.inflate(layoutInflater)
15	}
16	override fun onCreate(savedInstanceState: Bundle?) {
17	super.onCreate(savedInstanceState)
18	enableEdgeToEdge()
19	setContentView(binding.root)
20	ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.ma
21	in)) { v, insets ->
22	val systemBars =
23	insets.getInsets(WindowInsetsCompat.Type.systemBars())
24	v.setPadding(systemBars.left, systemBars.top,
25	systemBars.right, systemBars.bottom)
26	insets
27	}
28	setSupportActionBar(binding.myToolbar)
29	val navHostFragment = supportFragmentManager
30	.findFragmentById(R.id.fragmentContainerView) as
31	NavHostFragment
32	val navController = navHostFragment.navController
33	
34	setupActionBarWithNavController(navController)
35	}
36	override fun onSupportNavigateUp(): Boolean {
37	val navHostFragment = supportFragmentManager
	.findFragmentById(R.id.fragmentContainerView) as
	NavHostFragment
	return navHostFragment.navController.navigateUp()
	super.onSupportNavigateUp()
	}
	}

Model

Tabel 2. Source Code Soal 1 Film.kt XML

1	package com.example.filafixml.Model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class Film (
7	val id: Int,
8	@StringRes val title: Int,
9	@StringRes val synopsis: Int,
10	@StringRes val rating: Int,
11	@StringRes val review: Int,
12	@StringRes val score: Int,
13	@DrawableRes val imageId: Int,

14	@StringRes val imdbUrl: Int,
15	@DrawableRes val bigImageId: Int
16	)

ViewModel

FilmViewModel

Tabel 3. Source Code Soal 1 FilmViewModel.kt XML

1	package com.example.filafixml.viewmodel
2	
3	import androidx.lifecycle.LiveData
4	import androidx.lifecycle.MutableLiveData
5	import androidx.lifecycle.ViewModel
6	import com.example.filafixml.data.DataSource
7	import com.example.filafixml.model.Film
8	import kotlinx.coroutines.flow.MutableStateFlow
9	import kotlinx.coroutines.flow.StateFlow
10	import kotlinx.coroutines.flow.asStateFlow
11	import timber.log.Timber
12	
13	class FilmViewModel(
14	private val dataSource : DataSource,
15	val pageInfo : String
16	): ViewModel() {
17	private val _movieList =
18	MutableStateFlow<List<Film>>(emptyList())
19	private val _highlightList =
20	MutableStateFlow<List<Film>>(emptyList())
21	val movieList: StateFlow<List<Film>> =
22	_movieList.asStateFlow()
23	val highlightList: StateFlow<List<Film>> =
24	_highlightList.asStateFlow()
25	init {
26	loadData()
27	}
28	fun loadData() {
29	val allFilm = dataSource.listFilm()
30	_movieList.value = allFilm
31	Timber.d("Data Item yang Masuk Pada List : \${allFilm.size}")

32	<code>_highlightList.value = allFilm</code>
33	<code>}</code>
34	<code>}</code>

FilmViewModelFactory

Tabel 4. Source Code Soal 1 FilmViewModelFactory.kt XML

1	<code>package com.example.filafixml.viewmodel</code>
2	
3	<code>import androidx.lifecycle.ViewModel</code>
4	<code>import androidx.lifecycle.ViewModelProvider</code>
5	<code>import com.example.filafixml.data.DataSource</code>
6	
7	<code>class FilmViewModelFactory(</code>
8	<code> private val dataSource: DataSource,</code>
9	<code> private val pageInfo : String</code>
10	<code>): ViewModelProvider.Factory{</code>
11	<code> override fun <T : ViewModel> create(modelClass:</code>
12	<code>Class<T>): T {</code>
13	<code> if</code>
14	<code>(modelClass.isAssignableFrom(FilmViewModel::class.java)){</code>
15	<code> return FilmViewModel(dataSource, pageInfo) as</code>
16	<code>T</code>
17	<code> }</code>
18	<code> throw IllegalArgumentException("Unknown ViewModel</code>
19	<code>Class")</code>
20	<code> }</code>
21	<code>}</code>

DetailViewModel

Tabel 5. Source Code Soal 1 DetailViewModel.kt XML

1	<code>package com.example.filafixml.viewmodel</code>
2	
3	<code>import androidx.lifecycle.ViewModel</code>
4	<code>import com.example.filafixml.data.DataSource</code>
5	
6	<code>class DetailViewModel (</code>
7	<code> private val dataSource: DataSource,</code>
8	<code> val filmId : Int,</code>
9	<code> val pageInfo : String</code>
10	<code>): ViewModel(){</code>
11	<code> val film = dataSource.listFilm().find { it.id==</code>
12	<code>filmId }</code>
13	<code>}</code>

DetailViewModelFactory

Tabel 6. Source Code Soal 1 DetailViewModelFactory.kt XML

1	package com.example.filafixml.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import com.example.filafixml.data.DataSource
6	
7	class DetailViewModelFactory(8 private val dataSource: DataSource, 9 private val filmId: Int, 10 private val pageInfo: String 11): ViewModelProvider.Factory { 12 override fun <T : ViewModel> create(modelClass: 13 Class<T>): T { 14 if 15 (modelClass.isAssignableFrom(DetailViewModel::class.java 16)){ 17 return DetailViewModel(dataSource, filmId, 18 pageInfo) as T 19 } 20 throw IllegalArgumentException("Unknown 21 ViewModel Class") 22 } 23 } 24 }

LanguageViewModel

Tabel 7. Source Code Soal 1 LanguageViewModel.kt XML

1	package com.example.filafixml.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	
5	class LanguageViewModel(val pageInfo : String): ViewModel() {}

LanguageViewModelFactory

Tabel 8. Source Code Soal 1 LanguageViewModelFactory.kt XML

1	package com.example.filafixml.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class LanguageViewModelFactory (
7	private val pageInfo : String
8	): ViewModelProvider.Factory{
9	override fun <T : ViewModel> create(modelClass:
10	Class<T>): T {
11	if(modelClass.isAssignableFrom(LanguageViewModel::class.
12	java)){
13	return LanguageViewModel(pageInfo) as T
14	}
15	throw IllegalArgumentException("Unknown
	ViewModel Class")
	}
	}

Data

DataSource.kt

Tabel 9. Source Code Soal 1 DataSource.kt XML

1	package com.example.filafixml.Data
2	
3	import com.example.filafixml.Model.Film
4	import com.example.filafixml.R
5	
6	class DataSource {
7	fun listFilm(): List<Film> {
8	return listOf(
9	Film(
10	1,
11	R.string.title_f01,
12	R.string.synopsis_f01,
13	R.string.rating,
14	R.string.review_f01,
15	R.string.score_f01,
16	R.drawable.sotl,
17	R.string.imdb_f01,
18	R.drawable.sotl2
19	),
20	Film(

21	2,
22	R.string.title_f02,
23	R.string.synopsis_f02,
24	R.string.rating,
25	R.string.review_f02,
26	R.string.score_f02,
27	R.drawable.gitc,
28	R.string.imdb_f02,
29	R.drawable.gitc2
30	),
31	Film(
32	3,
33	R.string.title_f03,
34	R.string.synopsis_f03,
35	R.string.rating,
36	R.string.review_f03,
37	R.string.score_f03,
38	R.drawable.phm,
39	R.string.imdb_f03,
40	R.drawable.phm
41	),
42	Film(4,
43	R.string.title_f04,
44	R.string.synopsis_f04,
45	R.string.rating,
46	R.string.review_f04,
47	R.string.score_f04,
48	R.drawable.sktp,
49	R.string.imdb_f04,
50	R.drawable.moko
51	),
52	Film(
53	5,
54	R.string.title_f05,
55	R.string.synopsis_f05,
56	R.string.rating,
57	R.string.review_f05,
58	R.string.score_f05,
59	R.drawable.wudm,
60	R.string.imdb_f05,
61	R.drawable.wudp2
62	)
63	)
64	}
65	}

Adapter

CarouselAdapter.kt

Tabel 10. Source Code Soal 1 CarouselAdapter XML

```
1 package com.example.filafixml.Adapter
2
3 import android.view.LayoutInflater
4 import android.view.ViewGroup
5 import androidx.recyclerview.widget.DiffUtil
6 import androidx.recyclerview.widget.ListAdapter
7 import androidx.recyclerview.widget.RecyclerView
8 import com.example.filafixml.Model.Film
9 import
10 com.example.filafixml.databinding.ItemHighlightBinding
11
12 class CarouselAdapter(private val onItemClick: (Film)-
13 >Unit) :
14     ListAdapter<Film,
15     CarouselAdapter.CarouselViewHolder>(DiffCallback) {
16
17     override fun onCreateViewHolder(parent: ViewGroup,
18     viewType: Int): CarouselViewHolder{
19         val binding =
20         ItemHighlightBinding.inflate(LayoutInflater.from(parent.
21     context), parent, false)
22         return CarouselViewHolder(binding)
23     }
24
25     override fun onBindViewHolder(holder:
26     CarouselViewHolder, position : Int) {
27         holder.bind(getItem(position))
28     }
29
30     inner class CarouselViewHolder(private val binding:
31     ItemHighlightBinding):
32         RecyclerView.ViewHolder(binding.root) {
33         fun bind (film : Film){
34             binding.apply {
35                 ivHighlight.setImageResource(film.imageId)
36                 root.setOnClickListener {
37                     onItemClick(film) }
38             }
39         }
40     }
41
42     companion object DiffCallback :
43     DiffUtil.ItemCallback<Film>() {
44         override fun areItemsTheSame(oldItem: Film,
45         newItem: Film): Boolean = oldItem.id == newItem.id
46         override fun areContentsTheSame(oldItem: Film,
47         newItem: Film): Boolean = oldItem == newItem
48     }
```

35	<pre> } } </pre>
----	------------------------------

MovieListAdapter.kt

Tabel 11. Source Code Soal 1 MovieListAdapter XML

1	package com.example.filafixml.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.DiffUtil
6	import androidx.recyclerview.widget.ListAdapter
7	import androidx.recyclerview.widget.RecyclerView
8	import com.example.filafixml.model.Film
9	import com.example.filafixml.databinding.ItemFilmBinding
10	import timber.log.Timber
11	
12	class MovieListAdapter(
13	private val onImdbClick: (Film) -> Unit,
14	private val onDetailsClick: (Film)->Unit):
15	ListAdapter<Film,
	MovieListAdapter.MovieViewHolder>(DiffCallback){
16	override fun onCreateViewHolder(parent: ViewGroup,
	viewType: Int): MovieViewHolder {
17	val binding =
	ItemFilmBinding.inflate(LayoutInflater.from(parent.conte
	xt), parent, false)
18	return MovieViewHolder(binding)
19	}
20	override fun onBindViewHolder(holder:
	MovieViewHolder, position : Int){
21	val film = getItem(position)
22	holder.bind(film)
23	}
24	
25	inner class MovieViewHolder(private val binding:
	ItemFilmBinding) :
26	RecyclerView.ViewHolder(binding.root) {
27	fun bind(film: Film) {
28	binding.apply {
29	tvTitleFilm.text =
	root.context.getString(film.title)
30	tvRatingFilm.text =
	root.context.getString(film.rating)
31	ivImageFilm.setImageResource(film.imageId)

32	tvDescFilm.text=
	root.context.getString(film.synopsis)
33	tvScoreFilm.text=root.context.getString(film.score)
34	
35	root.setOnClickListener {
36	Timber.d("Item List dari
	\${root.context.getString(film.title)} Ditekan")
37	onDetailsClick(film)
38	}
39	btDetails.setOnClickListener {
40	Timber.d("Tombol Detail dari Film:
	\${root.context.getString(film.title)} Ditekan")
41	onDetailsClick(film)
42	}
43	btIMDB.setOnClickListener {
44	Timber.d("Tombol IMDB dari Film :
	\${root.context.getString(film.title)} Ditekan")
45	onImdbClick(film)
46	}
47	}
48	}
49	}
50	companion object DiffCallback :
	DiffUtil.ItemCallback<Film>() {
51	override fun areItemsTheSame(oldItem: Film,
	newItem: Film): Boolean = oldItem.id == newItem.id
52	override fun areContentsTheSame(oldItem: Film,
	newItem: Film): Boolean = oldItem == newItem
53	}
54	}

Fragment

HomeFragment.kt

Tabel 12. Source Code Soal HomeFragment.kt XML

1	package com.example.filafixml.fragment
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.View
7	import android.widget.Toast
8	import androidx.fragment.app.Fragment
9	import androidx.fragment.app.viewModels
10	import androidx.lifecycle.Lifecycle

```

11 import androidx.lifecycle.lifecycleScope
12 import androidx.lifecycle.repeatOnLifecycle
13 import com.example.filafixml.R
14 import androidx.navigation.fragment.findNavController
15 import com.example.filafixml.adapter.CarouselAdapter
16 import com.example.filafixml.adapter.MovieListAdapter
17 import com.example.filafixml.data.DataSource
18 import com.example.filafixml.viewmodel.FilmViewModel
19 import
    com.example.filafixml.databinding.FragmenthomeBinding
20 import
    com.example.filafixml.viewmodel.FilmViewModelFactory
21 import kotlinx.coroutines.launch
22 import timber.log.Timber
23 import kotlin.getValue
24
25 class HomeFragment : Fragment(R.layout.fragmenthome){
26     private val viewModel : FilmViewModel by viewModels
    {
27         FilmViewModelFactory(DataSource(),
    getString(R.string.homepage))
28     }
29
30     override fun onCreateView(view: View,
    savedInstanceState: Bundle?) {
31         super.onCreateView(view, savedInstanceState)
32         val binding = FragmenthomeBinding.bind(view)
33
34         val carouselAdapter = CarouselAdapter { film ->
35             val action =
    HomeFragmentDirections.actionHomeFragmentToDetailFragmen
    t(film.id)
36             findNavController().navigate(action)
37         }
38
39         val movieListAdapter = MovieListAdapter(
40             onDetailsClick = { film ->
41                 val action =
    HomeFragmentDirections.actionHomeFragmentToDetailFragmen
    t(film.id)
42                 findNavController().navigate(action)
43             }, onImdbClick = { film ->
44                 val url = getString(film.imdbUrl)
45                 val intent = Intent(Intent.ACTION_VIEW,
    Uri.parse(url))
46                 startActivity(intent)
47             }
48         )
49

```


50	binding.rvCarousel.apply {
51	adapter = carouselAdapter
52	layoutManager =
	androidx.recyclerview.widget.LinearLayoutManager(require
	Context()),
	androidx.recyclerview.widget.LinearLayoutManager.HORIZONTAL,
	false)
53	}
54	binding.rvMovieList.apply {
55	adapter = movieListAdapter
56	layoutManager =
	androidx.recyclerview.widget.LinearLayoutManager(require
	Context())
57	}
58	binding.btnLanguage.setOnClickListener {
59	val action =
	HomeFragmentDirections.actionHomeFragmentToLanguageFragment()
60	findNavController().navigate(action)
61	}
62	
63	viewLifecycleOwner.lifecycleScope.launch {
64	viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
65	launch {
66	viewModel.highlightList.collect {
	films ->
67	carouselAdapter.submitList(films)
68	}
69	}
70	launch {
71	viewModel.movieList.collect { films
	->
72	if(films.isNotEmpty()) {
73	Timber.d("Item Film yang
	Dikirimkan pada List: \${films.size}")
74	movieListAdapter.submitList(films)
75	}
76	}
77	}
78	}
79	}
80	
81	Toast.makeText(requireContext(),
	getString(R.string.homepage), Toast.LENGTH_SHORT).show()
82	

83	}
84	}

Detailfragment.kt

Tabel 13. Source Code Soal 1 DetailFragment XML

1	package com.example.filafixml.fragment
2	
3	import android.os.Bundle
4	import android.view.View
5	import android.widget.Toast
6	import androidx.fragment.app.Fragment
7	import androidx.fragment.app.viewModels
8	import androidx.navigation.fragment.findNavController
9	import androidx.navigation.fragment.navArgs
10	import com.example.filafixml.data.DataSource
11	import kotlin.getValue
12	import com.example.filafixml.R
13	import
	com.example.filafixml.databinding.FragmentdetailBinding
14	import com.example.filafixml.viewmodel.DetailViewModel
15	import
	com.example.filafixml.viewmodel.DetailViewModelFactory
16	import timber.log.Timber
17	
18	class DetailFragment: Fragment(R.layout.fragmentdetail)
	{
19	private val args: DetailFragmentArgs by navArgs()
20	private val viewModel : DetailViewModel by
	viewModels {
21	DetailViewModelFactory(
22	DataSource(),
23	args.filmId,
24	getString(R.string.detailpage)
25	)
26	}
27	
28	override fun onCreateView(view: View,
	savedInstanceState: Bundle?) {
29	super.onCreateView(view, savedInstanceState)
30	val binding = FragmentdetailBinding.bind(view)
31	
32	val film = viewModel.film
33	if (film != null){
34	Timber.d("Menampilkan Data Detail dari Film:
	`\${getString(film.title)}`")
35	binding.apply {

36	tvFilmTitle.text=getString(film.title)
37	tvFilmSynopsis.text=getString((film.synopsis))
38	tvFilmRating.text=getString(film.rating)
39	tvfilmScore.text=getString(film.score)
40	tvFilmReview.text=getString(R.string.review)
41	tvFilmComment.text=getString(film.review)
42	ivPoster.setImageResource(film.imageId)
43	headerBackDetail.btnBack.setOnClickListener { findNavController().navigateUp() }
44	ivBigImage.setImageResource(film.bigImageId)
45	}
46	Toast.makeText(requireContext(), viewModel.pageInfo, Toast.LENGTH_SHORT).show()
47	} else {
48	Toast.makeText(requireContext(), getString(R.string.errormsg), Toast.LENGTH_SHORT).show()
49	findNavController().navigateUp()
50	}
51	}
52	}

LanguageFragment.kt

Tabel 14. Source Code Soal 1 LanguageFragment XML

1	package com.example.filafixml.fragment
2	
3	import android.os.Bundle
4	import android.view.View
5	import android.widget.Toast
6	import androidx.appcompat.app.AppCompatActivity
7	import androidx.core.os.LocaleListCompat
8	import androidx.fragment.app.Fragment
9	import androidx.fragment.app.viewModels
10	import androidx.navigation.fragment.findNavController
11	import com.example.filafixml.R
12	import com.example.filafixml.databinding.FragmentlanguageBindin g
13	import com.example.filafixml.viewmodel.LanguageViewModel
14	import com.example.filafixml.viewmodel.LanguageViewModelFactory
15	

16	class LanguageFragment :
17	Fragment(R.layout.fragmentlanguage) {
	private val viewModel : LanguageViewModel by
	viewModels {
18	LanguageViewModelFactory(getString(R.string.languagepage
	))
19	}
20	override fun onCreateView(view: View,
	savedInstanceState: Bundle?) {
21	super.onCreateView(view, savedInstanceState)
22	val binding = FragmentlanguageBinding.bind(view)
23	
24	binding.rbEnglish.setOnClickListener {
	changeLanguage("en") }
25	binding.rbIndonesia.setOnClickListener {
	changeLanguage("in") }
26	
27	binding.headerBackLanguage.btnBack.setOnClickListener {
	findNavController().navigateUp() }
28	
29	Toast.makeText(requireContext(),
	viewModel.pageInfo, Toast.LENGTH_SHORT).show()
30	}
31	
32	fun changeLanguage(languageCode : String){
33	val appLocale : LocaleListCompat =
	LocaleListCompat.forLanguageTags(languageCode)
34	AppCompatActivity.setApplicationLocales(appLocale)
35	}
36	
37	}

Activity_Main.XML

Tabel 15. Source Code Soal 1 Activity_Main.XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity">
9	

10	<androidx.appcompat.widget.Toolbar
11	android:id="@+id/my_toolbar"
12	android:layout_width="match_parent"
13	android:layout_height="?attr/actionBarSize"
14	android:background="?attr/colorPrimary"
15	app:layout_constraintTop_toTopOf="parent" />
16	
17	<androidx.fragment.app.FragmentContainerView
18	android:id="@+id/fragmentContainerView"
19	android:name="androidx.navigation.fragment.NavHostFragment"
20	android:layout_width="0dp"
21	android:layout_height="0dp"
22	app:defaultNavHost="true"
23	app:navGraph="@navigation/nav_graph"
24	app:layout_constraintBottom_toBottomOf="parent"
25	app:layout_constraintEnd_toEndOf="parent"
26	app:layout_constraintStart_toStartOf="parent"
27	app:layout_constraintTop_toTopOf="parent" />
28	
29	</androidx.constraintlayout.widget.ConstraintLayout>

Fragmenthome.xml

Tabel 16. Source Code Soal 1 Fragmenthome.XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	xmlns:app="http://schemas.android.com/apk/res-auto">
7	
8	<com.google.android.material.card.MaterialCardView
9	android:id="@+id/headerCard"
10	android:layout_width="match_parent"
11	android:layout_height="wrap_content"
12	android:backgroundTint="@color/headercard"
13	app:cardCornerRadius="0dp"
14	app:layout_constraintTop_toTopOf="parent"
15	app:layout_constraintStart_toStartOf="parent"
16	app:layout_constraintEnd_toEndOf="parent">
17	
18	<androidx.constraintlayout.widget.ConstraintLayout
19	android:layout_width="match_parent"
20	android:layout_height="wrap_content"
21	android:padding="24dp">
22	
23	
24	<TextView
25	android:id="@+id/tvTitle"
26	android:layout_width="0dp"
27	android:layout_height="wrap_content"

```

28         android:text="@string/app_title"
29         android:textSize="24sp"
30         android:textStyle="bold"
31         android:layout_marginTop="5dp"
32         android:layout_marginStart="8dp"
33         app:layout_constraintStart_toStartOf="parent"
34         app:layout_constraintTop_toTopOf="parent"
35     />
36
37     <LinearLayout
38         android:layout_width="wrap_content"
39         android:layout_height="wrap_content"
40         android:orientation="vertical"
41         android:layout_marginTop="5dp"
42         app:layout_constraintTop_toTopOf="parent"
43         app:layout_constraintEnd_toEndOf="parent">
44
45         <ImageButton
46             android:id="@+id/btnLanguage"
47             android:layout_width="wrap_content"
48             android:layout_height="wrap_content"
49             android:layout_marginStart="16dp"
50             android:src="@drawable/btn_language"
51             android:background="@android:color/transparent"
52         />
53
54         <TextView
55             android:layout_width="wrap_content"
56             android:layout_height="wrap_content"
57             android:text="@string/languagebutton"
58             android:textStyle="bold"
59             android:layout_marginTop="2dp"
60         >
61
62         </TextView>
63
64     </LinearLayout>
65
66     </androidx.constraintlayout.widget.ConstraintLayout>
67
68     <androidx.core.widget.NestedScrollView
69         android:layout_width="match_parent"
70         android:layout_height="0dp"
71         app:layout_constraintTop_toBottomOf="@id/headerCard"
72         app:layout_constraintBottom_toBottomOf="parent"
73         android:fillViewport="true">
74
75         <androidx.constraintlayout.widget.ConstraintLayout
76             android:layout_width="match_parent"
77             android:layout_height="wrap_content">
78
79         <com.google.android.material.card.MaterialCardView
80             android:id="@+id/cvHighlight"

```

```

81         android:layout_width="match_parent"
82         android:layout_height="wrap_content"
83         android:layout_marginTop="8dp"
84         android:layout_marginStart="4dp"
85         android:layout_marginEnd="4dp"
86         android:backgroundTint="@color/highlightcard"
87         app:layout_constraintTop_toTopOf="parent">
88
89         <androidx.recyclerview.widget.RecyclerView
90             android:id="@+id/rvCarousel"
91             android:layout_width="match_parent"
92             android:layout_height="170dp"
93             android:orientation="horizontal">
94
95         </androidx.recyclerview.widget.RecyclerView>
96
97
98     </com.google.android.material.card.MaterialCardView>
99
100    <TextView
101        android:id="@+id/tvAboutApp"
102        android:layout_width="wrap_content"
103        android:layout_height="wrap_content"
104        android:text="@string/app_about"
105        android:textStyle="bold"
106        android:textSize="18sp"
107        android:layout_marginStart="4dp"
108        android:layout_marginTop="8dp"
109        app:layout_constraintTop_toBottomOf="@id/cvHighlight"
110        app:layout_constraintStart_toStartOf="parent">
111
112    </TextView>
113
114    <androidx.recyclerview.widget.RecyclerView
115        android:id="@+id/rvMovieList"
116        android:layout_width="match_parent"
117        android:layout_height="wrap_content"
118        android:layout_marginTop="8dp"
119        android:layout_marginStart="4dp"
120        android:layout_marginEnd="4dp"
121        app:layout_constraintTop_toBottomOf="@id/tvAboutApp"
122        app:layout_constraintStart_toStartOf="parent"
123        app:layout_constraintEnd_toEndOf="parent"
124    >
125
126    </androidx.recyclerview.widget.RecyclerView>
127
128    </androidx.constraintlayout.widget.ConstraintLayout>
129
130 </androidx.core.widget.NestedScrollView>
131
132
133
134

```

135	</androidx.constraintlayout.widget.ConstraintLayout>
-----	--

Fragmentdetail.xml

Tabel 17. Source Code Soal 1 Fragmentdetail.XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	xmlns:app="http://schemas.android.com/apk/res-auto"
7	xmlns:tools="http://schemas.android.com/tools">
8	
9	<include
10	android:id="@+id/headerBackDetail"
11	layout="@layout/header_back"
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"/>
14	
15	<androidx.core.widget.NestedScrollView
16	android:layout_width="match_parent"
17	android:layout_height="0dp"
18	android:fillViewport="true"
19	app:layout_constraintTop_toBottomOf="@id/headerBackDetail"
20	app:layout_constraintBottom_toBottomOf="parent">
21	
22	<androidx.constraintlayout.widget.ConstraintLayout
23	android:layout_width="match_parent"
24	android:layout_height="wrap_content">
25	
26	<com.google.android.material.card.MaterialCardView
27	android:id="@+id/cvBigImage"
28	android:layout_width="match_parent"
29	android:layout_height="200dp"
30	android:backgroundTint="@color/highlightcard"
31	app:cardCornerRadius="0dp"
32	app:cardPreventCornerOverlap="false"
33	app:layout_constraintTop_toTopOf="parent">
34	
35	<ImageView
36	android:id="@+id/ivBigImage"
37	android:layout_width="match_parent"
38	android:layout_height="wrap_content"
39	android:adjustViewBounds="true"
40	android:scaleType="centerCrop"
41	tools:src="@drawable/sotl2">
42	
43	</ImageView>
44	
45	</com.google.android.material.card.MaterialCardView>


```

46
47         <com.google.android.material.card.MaterialCardView
48             android:id="@+id/cvPoster"
49             android:layout_width="200dp"
50             android:layout_height="0dp"
51             android:layout_marginEnd="4dp"
52             android:layout_marginTop="8dp"
53             app:cardCornerRadius="16dp"
54             app:layout_constraintTop_toBottomOf="@id/cvBigImage"
55             app:layout_constraintEnd_toEndOf="parent"
56 app:layout_constraintBottom_toBottomOf="@id/tvFilmSynopsis"
57         >
58             <ImageView
59                 android:id="@+id/ivPoster"
60                 android:layout_width="match_parent"
61                 android:layout_height="match_parent"
62                 tools:src="@drawable/sotl"
63                 android:adjustViewBounds="true"
64                 android:scaleType="centerCrop"
65             >
66         </ImageView>
67
68
69 </com.google.android.material.card.MaterialCardView>
70
71         <TextView
72             android:id="@+id/tvFilmTitle"
73             android:layout_width="0dp"
74             android:layout_height="wrap_content"
75             android:layout_marginTop="8dp"
76             android:layout_marginEnd="8dp"
77             android:layout_marginStart="4dp"
78             app:layout_constraintTop_toBottomOf="@id/cvBigImage"
79 app:layout_constraintEnd_toStartOf="@id/cvPoster"
80             app:layout_constraintStart_toStartOf="parent"
81             tools:text="@string/title_f01"
82             android:textStyle="bold"
83             android:textSize="24dp">
84
85         </TextView>
86
87         <TextView
88             android:id="@+id/tvFilmSynopsis"
89             android:layout_width="0dp"
90             android:layout_height="wrap_content"
91             android:textStyle="bold"
92             tools:text="@string/synopsis_f01"
93             android:layout_marginTop="8dp"
94             android:layout_marginStart="4dp"
95             android:layout_marginEnd="8dp"
96             app:layout_constraintTop_toBottomOf="@id/tvFilmTitle"
97             app:layout_constraintEnd_toStartOf="@id/cvPoster"
98             app:layout_constraintStart_toStartOf="parent"
99

```

```

100         </TextView>
101
102         <TextView
103             android:id="@+id/tvFilmRating"
104             android:layout_width="wrap_content"
105             android:layout_height="wrap_content"
106             tools:text="@string/rating"
107             android:textStyle="bold"
108             android:textSize="25sp"
109             android:layout_marginTop="12dp"
110             android:layout_marginStart="4dp"
111             app:layout_constraintTop_toBottomOf="@id/tvFilmSynopsis"
112             app:layout_constraintStart_toStartOf="parent"
113         >
114
115         </TextView>
116
117         <TextView
118             android:id="@+id/tvfilmScore"
119             android:layout_width="wrap_content"
120             android:layout_height="wrap_content"
121             tools:text="@string/score_f01"
122             android:textStyle="bold"
123             android:textSize="25sp"
124             android:layout_marginTop="12dp"
125             android:layout_marginStart="4dp"
126             app:layout_constraintStart_toEndOf="@id/tvFilmRating"
127             app:layout_constraintTop_toBottomOf="@id/tvFilmSynopsis">
128
129         </TextView>
130
131         <com.google.android.material.card.MaterialCardView
132             android:layout_width="match_parent"
133             android:layout_height="wrap_content"
134             android:backgroundTint="@color/headercard"
135             app:cardCornerRadius="16dp"
136             android:layout_margin="4dp"
137             android:padding="8dp"
138             app:layout_constraintTop_toBottomOf="@id/tvFilmRating"
139             app:layout_constraintStart_toStartOf="parent"
140             app:layout_constraintEnd_toEndOf="parent"
141             app:layout_constraintBottom_toBottomOf="parent">
142
143             <androidx.constraintlayout.widget.ConstraintLayout
144                 android:layout_width="match_parent"
145                 android:layout_height="wrap_content">
146
147                 <TextView
148                     android:id="@+id/tvFilmReview"
149                     android:layout_width="wrap_content"
150                     android:layout_height="wrap_content"
151                     tools:text="@string/review"
152                     android:textStyle="bold"
153                     android:textSize="18sp"

```

```

154         android:layout_marginTop="4dp"
155         android:layout_marginStart="4dp"
156         app:layout_constraintTop_toTopOf="parent"
157         app:layout_constraintStart_toStartOf="parent">
158
159         </TextView>
160
161         <TextView
162             android:id="@+id/tvFilmComment"
163             android:layout_width="0dp"
164             android:layout_height="wrap_content"
165             tools:text="@string/review_f01"
166             android:textSize="14sp"
167             android:layout_marginStart="4dp"
168             android:layout_marginEnd="8dp"
169             android:textAppearance="@style/TextAppearance.AppCompat.Body1"
170             android:layout_marginBottom="4dp"
171             app:layout_constraintTop_toBottomOf="@id/tvFilmReview"
172             app:layout_constraintStart_toStartOf="parent"
173             app:layout_constraintEnd_toEndOf="parent"
174             app:layout_constraintBottom_toBottomOf="parent">
175
176         </TextView>
177
178     </androidx.constraintlayout.widget.ConstraintLayout>
179
180 </com.google.android.material.card.MaterialCardView>
181 </androidx.constraintlayout.widget.ConstraintLayout>
182
183 </androidx.core.widget.NestedScrollView>
184
185
186 </androidx.constraintlayout.widget.ConstraintLayout>

```

Fragmentlanguage.xml

Tabel 18. Source Code Soal 1 Fragmentlanguage.XML

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4         android:layout_width="match_parent"
5         android:layout_height="match_parent"
6         xmlns:app="http://schemas.android.com/apk/res-auto">
7
8     <include
9         android:id="@+id/headerBackLanguage"
10        layout="@layout/header_back"
11        android:layout_width="match_parent"
12        android:layout_height="wrap_content"/>
13
14    <RadioGroup
15        android:id="@+id/rgLanguage"

```

16	android:layout_width="wrap_content"
17	android:layout_height="wrap_content"
18	android:orientation="vertical"
19	app:layout_constraintTop_toBottomOf="@id/headerBackLanguage"
20	app:layout_constraintStart_toStartOf="parent">
21	
22	<RadioButton
23	android:id="@+id/rbEnglish"
24	android:layout_width="wrap_content"
25	android:layout_height="wrap_content"
26	android:text="@string/english">
27	</RadioButton>
28	
29	<RadioButton
30	android:id="@+id/rbIndonesia"
31	android:layout_width="wrap_content"
32	android:layout_height="wrap_content"
33	android:text="@string/indonesia">
34	
35	</RadioButton>
36	
37	</RadioGroup>
38	
39	</androidx.constraintlayout.widget.ConstraintLayout>

Header_back.xml

Tabel 19. Source Code Soal 1 Header_back.XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	xmlns:app="http://schemas.android.com/apk/res-auto">
7	
8	<com.google.android.material.card.MaterialCardView
9	android:id="@+id/headerBack"
10	android:layout_width="match_parent"
11	android:layout_height="wrap_content"
12	android:backgroundTint="@color/headercard"
13	app:cardCornerRadius="0dp"
14	app:layout_constraintTop_toTopOf="parent"
15	app:layout_constraintStart_toStartOf="parent"
16	app:layout_constraintEnd_toEndOf="parent">
17	
18	<androidx.constraintlayout.widget.ConstraintLayout
19	android:layout_width="match_parent"
20	android:layout_height="wrap_content"
21	android:padding="24dp">
22	
23	

24	
25	<TextView
26	android:id="@+id/tvTitle"
27	android:layout_width="0dp"
28	android:layout_height="wrap_content"
29	android:text="@string/app_title"
30	android:textSize="24sp"
31	android:textStyle="bold"
32	android:layout_marginTop="5dp"
33	android:layout_marginStart="8dp"
34	app:layout_constraintStart_toStartOf="parent"
35	app:layout_constraintTop_toTopOf="parent"
36	/>
37	<LinearLayout
38	android:layout_width="wrap_content"
39	android:layout_height="wrap_content"
40	android:orientation="vertical"
41	android:layout_marginTop="5dp"
42	app:layout_constraintTop_toTopOf="parent"
43	app:layout_constraintEnd_toEndOf="parent">
44	
45	<ImageButton
46	android:id="@+id/btnBack"
47	android:layout_width="wrap_content"
48	android:layout_height="wrap_content"
49	android:layout_marginStart="8dp"
50	android:src="@drawable/home"
51	android:background="@android:color/transparent"
52	/>
53	
54	<TextView
55	android:layout_width="wrap_content"
56	android:layout_height="wrap_content"
57	android:text="@string/homebutton"
58	android:textStyle="bold"
59	android:layout_marginTop="2dp"
60	>
61	
62	</TextView>
63	
64	</LinearLayout>
65	</androidx.constraintlayout.widget.ConstraintLayout>
66	</com.google.android.material.card.MaterialCardView>
	</androidx.constraintlayout.widget.ConstraintLayout>

Item_film.xml

Tabel 20. Source Code Soal 1 Item_film.XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	android:layout_width="match_parent"

4	android:layout_height="wrap_content"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	xmlns:tools="http://schemas.android.com/tools">
7	
8	<com.google.android.material.card.MaterialCardView
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content"
11	android:backgroundTint="@color/headercard"
12	app:cardCornerRadius="16dp"
13	app:cardElevation="4dp"
14	app:cardUseCompatPadding="true"
15	app:layout_constraintTop_toTopOf="parent"
16	app:layout_constraintStart_toStartOf="parent"
17	app:layout_constraintEnd_toEndOf="parent"
18	>
19	
20	<androidx.constraintlayout.widget.ConstraintLayout
21	android:layout_width="match_parent"
22	android:layout_height="wrap_content"
23	android:padding="8dp">
24	
25	<com.google.android.material.card.MaterialCardView
26	android:id="@+id/cvFilm"
27	android:layout_width="wrap_content"
28	android:layout_height="200dp"
29	app:cardCornerRadius="16dp"
30	app:layout_constraintStart_toStartOf="parent"
31	app:layout_constraintTop_toTopOf="parent"
32	app:layout_constraintBottom_toBottomOf="parent"
33	>
34	
35	
36	<ImageView
37	android:id="@+id/ivImageFilm"
38	android:layout_width="150dp"
39	android:layout_height="match_parent"
40	tools:src="@drawable/sotl"
41	android:scaleType="centerCrop"
42	android:adjustViewBounds="true"
43	>
44	</ImageView>
45	</com.google.android.material.card.MaterialCardView>
46	
47	
48	<TextView
49	android:id="@+id/tvTitleFilm"
50	android:layout_width="0dp"
51	android:layout_height="wrap_content"
52	android:layout_marginStart="8dp"
53	android:layout_marginEnd="8dp"
54	tools:text="@string/title_f01"
55	android:textStyle="bold"
56	android:textSize="18sp"
57	app:layout_constraintTop_toTopOf="parent"

58	app:layout_constraintStart_toEndOf="@id/cvFilm"
59	app:layout_constraintEnd_toEndOf="parent"
60	>
61	</TextView>
62	
63	<TextView
64	android:id="@+id/tvDescFilm"
65	android:layout_width="wrap_content"
66	android:layout_height="wrap_content"
67	android:layout_marginTop="4dp"
68	android:layout_marginStart="8dp"
69	android:layout_marginEnd="8dp"
70	tools:text="@string/synopsis_f01"
71	android:maxLines="3"
72	android:ellipsize="end"
73	android:lineSpacingExtra="2dp"
74	android:hyphenationFrequency="normal"
75	android:textAlignment="viewStart"
76	app:layout_constrainedWidth="true"
77	app:layout_constraintTop_toBottomOf="@id/tvTitleFilm"
78	app:layout_constraintStart_toEndOf="@id/cvFilm"
79	app:layout_constraintEnd_toEndOf="parent"
80	>
81	
82	</TextView>
83	<TextView
84	android:id="@+id/tvRatingFilm"
85	android:layout_width="wrap_content"
86	android:layout_height="wrap_content"
87	android:textStyle="bold"
88	android:layout_marginStart="8dp"
89	android:layout_marginTop="4dp"
90	tools:text="@string/rating"
91	app:layout_constraintTop_toBottomOf="@id/tvDescFilm"
92	app:layout_constraintStart_toEndOf="@id/cvFilm"
93	>
94	
95	</TextView>
96	
97	<TextView
98	android:id="@+id/tvScoreFilm"
99	android:layout_width="wrap_content"
100	android:layout_height="wrap_content"
101	android:textStyle="bold"
102	android:layout_marginStart="4dp"
103	android:layout_marginTop="4dp"
104	tools:text="@string/score_f01"
105	app:layout_constraintTop_toBottomOf="@id/tvDescFilm"
106	app:layout_constraintStart_toEndOf="@id/tvRatingFilm"
107	>
108	
109	</TextView>
110	
111	<Button

```

112         android:id="@+id/btIMDB"
113         android:layout_width="82dp"
114         android:layout_height="wrap_content"
115         android:layout_marginTop="8dp"
116         android:layout_marginStart="8dp"
117         android:layout_marginEnd="2dp"
118         android:layout_marginBottom="12dp"
119         android:text="@string/imdb"
120         android:textColor="@color/black"
121         android:textStyle="bold"
122         android:textSize="13sp"
123         android:backgroundTint="@color/highlightcard"
124         app:layout_constraintHorizontal_weight="1"
125         app:layout_constraintStart_toEndOf="@id/cvFilm"
126         app:layout_constraintEnd_toStartOf="@id/btDetails"
127         app:layout_constraintTop_toBottomOf="@id/tvScoreFilm"
128         app:layout_constraintBottom_toBottomOf="parent"
129         app:layout_constraintHorizontal_chainStyle="spread">
130
131     </Button>
132
133     <Button
134         android:id="@+id/btDetails"
135         android:layout_width="82dp"
136         android:layout_height="wrap_content"
137         android:layout_marginTop="8dp"
138         android:layout_marginEnd="6dp"
139         android:layout_marginBottom="12dp"
140         android:text="@string/detail"
141         android:textColor="@color/black"
142         android:textStyle="bold"
143         android:textSize="13sp"
144         android:padding="6dp"
145         android:backgroundTint="@color/highlightcard"
146         app:layout_constraintHorizontal_weight="1"
147         app:layout_constraintStart_toEndOf="@id/btIMDB"
148         app:layout_constraintEnd_toEndOf="parent"
149         app:layout_constraintTop_toBottomOf="@id/tvScoreFilm"
150         app:layout_constraintBottom_toBottomOf="parent">
151
152     </Button>
153 </androidx.constraintlayout.widget.ConstraintLayout>
154 </com.google.android.material.card.MaterialCardView>
155
156
157
158
159 </androidx.constraintlayout.widget.ConstraintLayout>

```

Item_highlight

Tabel 21. Source Code Soal 1 item_highlight.XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	xmlns:app="http://schemas.android.com/apk/res-auto"
7	xmlns:tools="http://schemas.android.com/tools">
8	
9	<com.google.android.material.card.MaterialCardView
10	android:layout_width="match_parent"
11	android:layout_height="wrap_content"
12	app:cardCornerRadius="16dp"
13	app:cardElevation="4dp"
14	app:cardUseCompatPadding="false"
15	app:layout_constraintTop_toTopOf="parent"
16	app:layout_constraintStart_toStartOf="parent"
17	app:layout_constraintEnd_toEndOf="parent">
18	
19	<ImageView
20	android:id="@+id/ivHighlight"
21	android:layout_width="match_parent"
22	android:layout_height="match_parent"
23	tools:src="@drawable/sotl"
24	android:scaleType="centerCrop"
25	android:adjustViewBounds="true">
26	</ImageView>
27	
28	</com.google.android.material.card.MaterialCardView>
29	
30	</androidx.constraintlayout.widget.ConstraintLayout>

Navigation

Nav_graph

Tabel 22. Source Code Soal 1 nav_graph.XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/nav_graph"
6	app:startDestination="@id/homeFragment">
7	
8	<fragment
9	android:id="@+id/homeFragment"
10	android:name="com.example.filafixml.fragment.HomeFragment"
11	android:label="home"
12	tools:layout="@layout/fragmenthome">
13	

14	<action
15	android:id="@+id/action_homeFragment_to_detailFragment"
16	app:destination="@id/detailFragment"/>
17	<action
18	android:id="@+id/action_homeFragment_to_languageFragment"
19	app:destination="@id/languageFragment"/>
20	</fragment>
21	
22	<fragment
23	android:id="@+id/detailFragment"
24	android:name="com.example.filafixml.fragment.DetailFragment"
25	android:label="detail"
26	tools:layout="@layout/fragmentdetail">
27	
28	<argument
29	android:name="filmId"
30	app:argType="integer"/>
31	</fragment>
32	
33	<fragment
34	android:id="@+id/languageFragment"
35	android:name="com.example.filafixml.fragment.LanguageFragment"
36	android:label="language"
37	tools:layout="@layout/fragmentlanguage"/>
39	
39	</navigation>

Compose

MainActivity.kt

Tabel 23. Source Code Soal 1 MainActivity.kt Compose

1	package com.example.listcompose
2	
3	import android.os.Bundle
4	import androidx.activity.compose.setContent
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.compose.material3.MaterialTheme
7	import androidx.compose.material3.Surface
8	import androidx.navigation.compose.rememberNavController
9	import com.example.listcompose.Navigation.Nav_Graph
10	import com.example.listcompose.ui.theme.ListComposeTheme
11	
12	class MainActivity : AppCompatActivity() {
13	override fun onCreate(savedInstanceState: Bundle?) {
14	super.onCreate(savedInstanceState)
15	setContent {

16	ListComposeTheme {
17	val navController = rememberNavController()
18	Surface(
19	color =
	MaterialTheme.colorScheme.background
20	) {
21	Nav_Graph(
22	navController = navController,
23	startDestination = "home"
24	)
25	}
26	}
27	}
28	}
29	}
30	

Model

Film.kt

Tabel 24. Source Code Soal 1 Film.kt Compose

1	package com.example.listcompose.Model
2	
3	data class Film (
4	val id: Int,
5	val title: Int,
6	val synopsis: Int,
7	val rating: Int,
8	val review: Int,
9	val score: Int,
10	val imageId: Int,
11	val imdbUrl: Int,
12	val bigImageId: Int
13	)

Data

DataSource.kt

Tabel 25. Source Code Soal 1 DataSource.kt Compose

1	package com.example.listcompose.Data
2	
3	import com.example.listcompose.Model.Film
4	import com.example.listcompose.R
5	
6	class DataSource {
7	fun listFilm(): List<Film> {
8	return listOf(

9	Film(
10	1,
11	R.string.title_f01,
12	R.string.synopsis_f01,
13	R.string.rating,
14	R.string.review_f01,
15	R.string.score_f01,
16	R.drawable.sotl,
17	R.string.imdb_f01,
18	R.drawable.sotl2
19	),
20	Film(
21	2,
22	R.string.title_f02,
23	R.string.synopsis_f02,
24	R.string.rating,
25	R.string.review_f02,
26	R.string.score_f02,
27	R.drawable.gitc,
28	R.string.imdb_f02,
29	R.drawable.gitc2
30	),
31	Film(
32	3,
33	R.string.title_f03,
34	R.string.synopsis_f03,
35	R.string.rating,
36	R.string.review_f03,
37	R.string.score_f03,
38	R.drawable.phm,
39	R.string.imdb_f03,
40	R.drawable.phm
41	),
42	Film(4,
43	R.string.title_f04,
44	R.string.synopsis_f04,
45	R.string.rating,
46	R.string.review_f04,
47	R.string.score_f04,
48	R.drawable.sktp,
49	R.string.imdb_f04,
50	R.drawable.sktp2
51	),
52	Film(
53	5,
54	R.string.title_f05,
55	R.string.synopsis_f05,
56	R.string.rating,
57	R.string.review_f05,
58	R.string.score_f05,
59	R.drawable.wudm,
60	R.string.imdb_f05,
61	R.drawable.wudm2
62	)

63	)
64	}
65	}

ViewModel

FilmViewModel

Tabel 26. Source Code Soal 1 FilmViewModel.kt Compose

1	package com.example.listcompose.ViewModel
2	
3	import androidx.compose.runtime.mutableStateOf
4	import androidx.lifecycle.ViewModel
5	import com.example.listcompose.Data.DataSource
6	import com.example.listcompose.Model.Film
7	import androidx.compose.runtime.State
8	import kotlinx.coroutines.flow.MutableStateFlow
9	import kotlinx.coroutines.flow.StateFlow
10	import kotlinx.coroutines.flow.asStateFlow
11	
12	class FilmViewModel(
13	private val dataSource: DataSource,
14	val pageInfo: String
15	) : ViewModel() {
16	private val _films = MutableStateFlow(dataSource.listFilm())
17	val films: StateFlow<List<Film>> = _films.asStateFlow()
18	}

FilmViewModelFactory

Tabel 27. Source Code Soal 1 FilmViewModelFactory.kt Compose

1	package com.example.listcompose.ViewModel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import com.example.listcompose.Data.DataSource
6	
7	class FilmViewModelFactory(
8	private val dataSource: DataSource,
9	private val pageInfo: String
10	): ViewModelProvider.Factory {
11	override fun <T : ViewModel> create(modelClass: Class<T>): T
12	{
13	if(modelClass.isAssignableFrom(FilmViewModel::class.java)){
14	return FilmViewModel(dataSource, pageInfo) as T
14	}

15	throw IllegalArgumentException("Unknown ViewModel
16	Class")
17	}

DetailViewModel

Tabel 28. Source Code Soal 1 DetailViewModel.kt Compose

1	package com.example.listcompose.ViewModel
2	
3	import androidx.lifecycle.ViewModel
4	import com.example.listcompose.Data.DataSource
5	import com.example.listcompose.Model.Film
6	import kotlinx.coroutines.flow.MutableStateFlow
7	import kotlinx.coroutines.flow.StateFlow
8	import kotlinx.coroutines.flow.asStateFlow
9	
10	class DetailViewModel(
11	private val dataSource: DataSource,
12	val filmId: Int,
13	val pageInfo: String
14	): ViewModel() {
15	private val _films = MutableStateFlow(dataSource.listFilm())
16	val films: StateFlow<List<Film>> = _films.asStateFlow()
17	fun getFilmById(filmId: Int): Film? {
18	return _films.value.find { it.id == filmId }
19	}
20	}

DetailViewModelFactory

Tabel 29. Source Code Soal 1 DetailViewModelFactory.kt Compose

1	package com.example.listcompose.ViewModel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import com.example.listcompose.Data.DataSource
6	
7	class DetailViewModelFactory(
8	private val dataSource: DataSource,
9	private val filmId : Int,
10	private val pageInfo : String
11	): ViewModelProvider.Factory {
12	override fun <T : ViewModel> create(modelClass: Class<T>): T
13	{
14	if(modelClass.isAssignableFrom(DetailViewModel::class.java)){
15	return DetailViewModel(dataSource, filmId, pageInfo)
16	}
17	as T

15	}
16	throw IllegalArgumentException("Unknown ViewModel
17	Class")
17	}
18	}

LanguageViewModel

Tabel 30. Source Code Soal 1 LanguageViewModel .kt Compose

1	package com.example.listcompose.ViewModel
2	
3	import androidx.lifecycle.ViewModel
4	
5	class LanguageViewModel(val pageInfo: String) : ViewModel() {}

LanguageViewModelFactory

Tabel 31. Source Code Soal 1 LanguageViewModelFactory .kt Compose

1	package com.example.listcompose.ViewModel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class LanguageViewModelFactory(
7	private val pageInfo : String
8	): ViewModelProvider.Factory {
9	override fun <T : ViewModel> create(modelClass: Class<T>): T
10	{
11	if(modelClass.isAssignableFrom(LanguageViewModel::class.java)) {
12	return LanguageViewModel(pageInfo) as T
13	}
14	throw IllegalArgumentException("Unknown ViewModel
14	Class")
14	}
15	}

layout

HeaderScreen.kt

Tabel 32. Source Code Soal 1 HeaderScreen.kt Compose

```

1 package com.example.listcompose.Layout
2
3 import androidx.annotation.DrawableRes
4 import androidx.compose.foundation.clickable
5 import androidx.compose.foundation.layout.Arrangement
6 import androidx.compose.foundation.layout.Column
7 import androidx.compose.foundation.layout.Row
8 import androidx.compose.foundation.layout.fillMaxWidth
9 import androidx.compose.foundation.layout.padding
10 import androidx.compose.foundation.layout.size
11 import androidx.compose.material3.Icon
12 import androidx.compose.material3.MaterialTheme
13 import androidx.compose.material3.Surface
14 import androidx.compose.material3.Text
15 import androidx.compose.runtime.Composable
16 import androidx.compose.ui.Alignment
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.graphics.Color
19 import androidx.compose.ui.res.colorResource
20 import androidx.compose.ui.res.painterResource
21 import androidx.compose.ui.res.stringResource
22 import androidx.compose.ui.text.font.FontWeight
23 import androidx.compose.ui.tooling.preview.Preview
24 import androidx.compose.ui.unit.dp
25 import com.example.listcompose.R
26 import com.example.listcompose.ui.theme.ListComposeTheme
27
28 @Composable
29 fun HeaderScreen(
30     title : String,
31     buttonText : String,
32     @DrawableRes icon : Int,
33     onClick : ()->Unit
34 ) {
35     Surface(
36         modifier = Modifier.fillMaxWidth(),
37         color= colorResource(id = R.color.card),
38         shadowElevation = 4.dp
39     ) {
40         Row(
41             modifier = Modifier
42                 .padding(horizontal = 24.dp, vertical = 20.dp)
43                 .fillMaxWidth(),
44             horizontalArrangement = Arrangement.SpaceBetween,
45             verticalAlignment = Alignment.CenterVertically
46         ) {
47             Text(
48                 text = title,
49                 style =
50                     MaterialTheme.typography.headlineMedium.copy(
51                         fontWeight = FontWeight.Bold,
52                         color = Color.Black
53                     )
54             )
55         }
56     }
57 }

```



```

53         )
54         Column(
55             horizontalAlignment =
Alignment.CenterHorizontally,
56             modifier = Modifier.clickable { onClick()
57         }
58         ) {
59             Icon(
60                 painter = painterResource(id = icon),
61                 contentDescription = null,
62                 modifier = Modifier.size(28.dp),
63                 tint = Color.Unspecified
64             )
65             Text(
66                 text = buttonText,
67                 style =
MaterialTheme.typography.labelMedium.copy(
                    fontWeight = FontWeight.Bold
68                 )
69             )
70         }
71     }
72 }
73 }
74
75 @Preview(showBackground = true)
76 @Composable
77 fun HeaderPreview() {
78     ListComposeTheme {
79         HeaderScreen(title = stringResource(id =
80 R.string.app_title),
            buttonText = stringResource(id =
81 R.string.languagebutton),
            icon = R.drawable.language,
82             onClick = {})
83     }
84 }

```

HomeScreen

Tabel 33. Source Code Soal 1 HomeScreen.kt Compose

```

1 package com.example.listcompose.Layout
2
3 import android.content.Intent
4 import android.net.Uri
5 import android.widget.Toast
6 import androidx.compose.foundation.Image
7 import androidx.compose.foundation.layout.Arrangement
8 import androidx.compose.foundation.layout.Column
9 import androidx.compose.foundation.layout.PaddingValues
10 import androidx.compose.foundation.layout.Row

```

```

11 import androidx.compose.foundation.layout.Spacer
12 import androidx.compose.foundation.layout.fillMaxSize
13 import androidx.compose.foundation.layout.fillMaxWidth
14 import androidx.compose.foundation.layout.height
15 import androidx.compose.foundation.layout.padding
16 import androidx.compose.foundation.layout.size
17 import androidx.compose.foundation.layout.width
18 import androidx.compose.foundation.lazy.LazyColumn
19 import androidx.compose.foundation.lazy.LazyRow
20 import androidx.compose.foundation.lazy.items
21 import androidx.compose.foundation.shape.RoundedCornerShape
22 import androidx.compose.material3.Button
23 import androidx.compose.material3.ButtonDefaults
24 import androidx.compose.material3.Card
25 import androidx.compose.material3.CardDefaults
26 import androidx.compose.material3.MaterialTheme
27 import com.example.listcompose.R
28 import androidx.compose.material3.Scaffold
29 import androidx.compose.material3.Text
30 import androidx.compose.runtime.Composable
31 import androidx.compose.runtime.LaunchedEffect
32 import androidx.compose.runtime.getValue
33 import androidx.compose.ui.Alignment
34 import androidx.compose.ui.Modifier
35 import androidx.compose.ui.draw.clip
36 import androidx.compose.ui.graphics.Color
37 import androidx.compose.ui.layout.ContentScale
38 import androidx.compose.ui.platform.LocalContext
39 import androidx.compose.ui.res.colorResource
40 import androidx.compose.ui.res.painterResource
41 import androidx.compose.ui.res.stringResource
42 import androidx.compose.ui.text.font.FontWeight
43 import androidx.compose.ui.text.style.TextOverflow
44 import androidx.compose.ui.unit.dp
45 import androidx.compose.ui.unit.sp
46 import androidx.lifecycle.compose.collectAsStateWithLifecycle
47 import androidx.lifecycle.viewmodel.compose.viewModel
48 import androidx.navigation.NavController
49 import com.example.listcompose.Data.DataSource
50 import com.example.listcompose.Model.Film
51 import com.example.listcompose.ViewModel.FilmViewModel
52 import com.example.listcompose.ViewModel.FilmViewModelFactory
53 import timber.log.Timber
54
55
56 @Composable
57 fun HomeScreen(
58     navController: NavController,
59     viewModel: FilmViewModel
60 ) {
61     val context=LocalContext.current
62     val films by viewModel.films.collectAsStateWithLifecycle()
63
64     LaunchedEffect(films) {

```

```

65         if (films.isNotEmpty()) {
66             Timber.d("Item Film yang Dikirimkan pada List:
        ${films.size}")
67         }
68     }
69
70     LaunchedEffect(Unit) {
71         val homepageText = context.getString(R.string.homepage)
72         Toast.makeText(context, homepageText,
        Toast.LENGTH_SHORT).show()
73     }
74     Scaffold(
75         topBar = {
76             HeaderScreen(
77                 title = stringResource(R.string.app_title),
78                 buttonText =
        stringResource(R.string.languagebutton),
79                 icon = R.drawable.language,
80                 onClick = {
        navController.navigate("language") }
81             )
82         }
83     ) { innerPadding ->
84         LazyColumn(
85             modifier = Modifier
86                 .fillMaxSize()
87                 .padding(innerPadding)
88         ) {
89             item {
90                 LazyRow(
91                     contentPadding = PaddingValues(16.dp),
92                     horizontalArrangement =
        Arrangement.spacedBy(12.dp)
93                 ) {
94                     items(films) { film ->
95                         HighlightCard(film)
96                     }
97                 }
98             }
99
100             item {
101                 Text(
102                     text = stringResource(R.string.app_about),
103                     style =
        MaterialTheme.typography.titleMedium.copy(fontWeight =
        FontWeight.Bold),
104                     modifier = Modifier.padding(start = 16.dp,
        bottom = 8.dp)
105                 )
106             }
107
108             items(films) { film ->
109                 MovieItem(
110                     film = film,

```

```

111         onDetailClick = {
112             Timber.d("Tombol Detail dari Film :
113             ${context.getString(film.title)} Ditekan")
114
115             navController.navigate("detail/${film.id}")
116             },
117             onIMDBClick = {
118                 Timber.d("Tombol IMDB dari Film :
119                 ${context.getString(film.title)} Ditekan")
120                 val intent = Intent(Intent.ACTION_VIEW,
121                 Uri.parse(context.getString(film.imdbUrl)))
122                 context.startActivity(intent)
123             }
124         )
125     }
126 }
127
128 @Composable
129 fun HighlightCard(film : Film) {
130     Card(
131         modifier = Modifier
132             .width(280.dp)
133             .height(160.dp),
134         shape = RoundedCornerShape(16.dp)
135     ) {
136         Image(
137             painter = painterResource(id = film.imageId),
138             contentDescription = null,
139             contentScale = ContentScale.Crop,
140             modifier = Modifier.fillMaxSize()
141         )
142     }
143 }
144
145 @Composable
146 fun MovieItem(
147     film: Film,
148     onDetailClick: () -> Unit,
149     onIMDBClick: () -> Unit
150 ) {
151     Card(
152         modifier = Modifier
153             .fillMaxWidth()
154             .padding(horizontal = 16.dp, vertical = 8.dp),
155         colors =
156         CardDefaults.cardColors(colorResource(R.color.card)),
157         shape = RoundedCornerShape(16.dp)
158     ) {
159         Row(
160             modifier = Modifier
161                 .padding(12.dp)
162                 .fillMaxWidth()

```

```

160         ) {
161             Image(
162                 painter = painterResource(id = film.imageId),
163                 contentDescription = null,
164                 modifier = Modifier
165                     .size(width = 110.dp, height = 160.dp)
166                     .clip(RoundedCornerShape(12.dp)),
167                 contentScale = ContentScale.Crop
168             )
169
170             Column(
171                 modifier = Modifier
172                     .padding(start = 12.dp)
173                     .weight(1f)
174             ) {
175                 Text(
176                     text = stringResource(film.title),
177                     style =
178                     MaterialTheme.typography.titleMedium.copy(fontWeight =
179                     FontWeight.Bold),
180                     maxLines = 2,
181                     overflow = TextOverflow.Ellipsis
182                 )
183
184                 Text(
185                     text = stringResource(film.synopsis),
186                     style = MaterialTheme.typography.bodySmall,
187                     maxLines = 3,
188                     overflow = TextOverflow.Ellipsis,
189                     modifier = Modifier.padding(vertical =
190                     4.dp)
191                 )
192
193                 Row(verticalAlignment =
194                 Alignment.CenterVertically) {
195                     Text(
196                         text = stringResource(R.string.rating),
197                         style =
198                         MaterialTheme.typography.bodySmall.copy(fontWeight =
199                         FontWeight.Bold)
200                     )
201                     Spacer(modifier = Modifier.width(4.dp))
202                     Text(
203                         text = stringResource(film.score),
204                         style =
205                         MaterialTheme.typography.bodySmall.copy(fontWeight =
206                         FontWeight.Bold),
207                         color = Color.DarkGray
208                     )
209                 }
210
211                 Spacer(modifier = Modifier.height(8.dp))
212
213                 Row(

```

206	modifier = Modifier.fillMaxWidth(),
207	horizontalArrangement =
	Arrangement.spacedBy(8.dp)
208	) {
209	Button(
210	onClick = onIMDBClick,
211	modifier = Modifier.weight(1f),
212	colors =
	ButtonDefaults.buttonColors(colorResource(R.color.button)),
213	contentPadding = PaddingValues(0.dp)
214	) {
215	Text(stringResource(R.string.imdb),
	fontSize = 11.sp, color = Color.Black)
215	}
216	Button(
217	onClick = onDetailClick,
218	modifier = Modifier.weight(1f),
219	colors =
	ButtonDefaults.buttonColors(colorResource(R.color.button)),
220	contentPadding = PaddingValues(0.dp)
221	) {
223	Text(stringResource(R.string.detail),
	fontSize = 11.sp, color = Color.Black)
224	}
225	}
226	}
227	}
228	}
229	}

DetailScreen.kte

Tabel 34. Source Code Soal 1 DetailScreen.kt Compose

1	package com.example.listcompose.Layout
2	
3	import android.widget.Toast
4	import androidx.compose.foundation.Image
5	import androidx.compose.foundation.layout.Box
6	import androidx.compose.foundation.layout.Column
7	import androidx.compose.foundation.layout.Row
8	import androidx.compose.foundation.layout.Spacer
9	import androidx.compose.foundation.layout.fillMaxHeight
10	import androidx.compose.foundation.layout.fillMaxSize
11	import androidx.compose.foundation.layout.fillMaxWidth
12	import androidx.compose.foundation.layout.height
13	import androidx.compose.foundation.layout.padding
14	import androidx.compose.foundation.layout.size
15	import androidx.compose.foundation.layout.width
16	import androidx.compose.foundation.rememberScrollState
17	import androidx.compose.foundation.shape.RoundedCornerShape
18	import androidx.compose.foundation.verticalScroll

```

19 import androidx.compose.material3.Card
20 import androidx.compose.material3.CardDefaults
21 import androidx.compose.material3.MaterialTheme
22 import androidx.compose.material3.Scaffold
23 import androidx.compose.material3.Text
24 import androidx.compose.runtime.Composable
25 import androidx.compose.runtime.LaunchedEffect
26 import androidx.compose.runtime.getValue
27 import androidx.compose.runtime.remember
28 import androidx.compose.ui.Modifier
29 import androidx.compose.ui.draw.clip
30 import androidx.compose.ui.layout.ContentScale
31 import androidx.compose.ui.res.colorResource
32 import androidx.compose.ui.res.painterResource
33 import androidx.compose.ui.res.stringResource
34 import androidx.compose.ui.text.font.FontWeight
35 import androidx.compose.ui.unit.dp
36 import androidx.compose.ui.unit.sp
37 import com.example.listcompose.ViewModel.FilmViewModel
38 import com.example.listcompose.R
39 import androidx.navigation.NavController
40 import androidx.compose.ui.Alignment
41 import androidx.compose.ui.platform.LocalContext
42 import androidx.lifecycle.compose.collectAsStateWithLifecycle
43 import androidx.lifecycle.viewmodel.compose.viewModel
44 import com.example.listcompose.Data.DataSource
45 import com.example.listcompose.ViewModel.DetailViewModel
46 import com.example.listcompose.ViewModel.DetailViewModelFactory
47 import timber.log.Timber
48
49 @Composable
50 fun DetailScreen(
51     filmId: Int,
52     navController: NavController,
53     viewModel: DetailViewModel
54 ) {
55     val context = LocalContext.current
56     val films by viewModel.films.collectAsStateWithLifecycle()
57     val film = remember (films) {
58         viewModel.getFilmById(filmId)
59     }
60
61     LaunchedEffect(film) {
62         film?.let{
63             Timber.d("Menampilkan Data Detail dari Film:
64             ${context.getString(it.title)}")
65         }
66
67         LaunchedEffect(Unit) {
68             Toast.makeText(context, viewModel.pageInfo,
69             Toast.LENGTH_SHORT).show()
70         }

```

```

71 Scaffold(
72     topBar = {
73         HeaderScreen(
74             title = stringResource(R.string.app_title),
75             buttonText =
stringResource(R.string.homebutton),
76             icon = R.drawable.home,
77             onClick = { navController.popBackStack()
}
78         )
79     }
80 ) { innerPadding ->
81     if (film != null) {
82         Column(
83             modifier = Modifier
84                 .fillMaxSize()
85                 .padding(innerPadding)
86                 .verticalScroll(rememberScrollState())
87         ) {
88             Image(
89                 painter = painterResource(id =
film.bigImageId),
90                 contentDescription = null,
91                 modifier = Modifier
92                     .fillMaxWidth()
93                     .height(220.dp),
94                 contentScale = ContentScale.Crop
95             )
96
97             Column(modifier = Modifier.padding(16.dp)) {
98                 Row(
99                     modifier = Modifier.fillMaxWidth(),
100                     verticalAlignment = Alignment.Bottom
101                 ) {
102                     Column(modifier = Modifier.weight(1f))
103                     {
104                         Text(
105                             text = stringResource(id =
film.title),
106                             style =
MaterialTheme.typography.headlineSmall.copy(fontWeight =
FontWeight.Bold)
107                         )
108                         Spacer(modifier =
Modifier.height(8.dp))
109                         Text(
110                             text = stringResource(id =
film.synopsis),
111                             style =
MaterialTheme.typography.bodyMedium,
112                             lineHeight = 20.sp
113                         )
114                     }

```



```

115         Spacer(modifier =
116         Modifier.width(12.dp))
117
118         Image(
119             painter = painterResource(id =
120             film.imageId),
121             contentDescription = null,
122             modifier = Modifier
123                 .size(width = 120.dp, height =
124                 180.dp)
125                 .clip(RoundedCornerShape(12.dp))
126                 .fillMaxHeight(),
127                 contentScale = ContentScale.Crop
128             )
129         )
130         Spacer(modifier = Modifier.height(16.dp))
131         Text(
132             text = "${stringResource(id =
133             R.string.rating)} ${stringResource(id = film.score)}",
134             style =
135             MaterialTheme.typography.titleLarge.copy(fontWeight =
136             FontWeight.Bold)
137         )
138         Spacer(modifier = Modifier.height(16.dp))
139         Card(
140             modifier = Modifier.fillMaxWidth(),
141             colors =
142             CardDefaults.cardColors(colorResource(R.color.card)),
143             shape = RoundedCornerShape(16.dp)
144         ) {
145             Column(modifier =
146             Modifier.padding(16.dp)) {
147                 Text(
148                     text = stringResource(id =
149                     R.string.review),
150                     style =
151                     MaterialTheme.typography.titleMedium.copy(fontWeight =
152                     FontWeight.Bold)
153                 )
154                 Spacer(modifier =
155                 Modifier.height(4.dp))
156                 Text(
157                     text = stringResource(id =
158                     film.review),
159                     style =
160                     MaterialTheme.typography.bodyMedium
161                 )
162             }
163         }

```

154	}
155	}
156	} else {
157	Box(
158	modifier = Modifier. <i>fillMaxSize</i> (),
159	contentAlignment = Alignment.Center) {
160	Text(text = stringResource(id =
161	R.string.errormsg))
162	}
163	}
164	}

LanguageScreen.kt

Tabel 35. Source Code Soal 1 LanguageScreen.kt Compose

1	package com.example.listcompose.Layout
2	
3	import android.widget.Toast
4	import androidx.core.os.LocaleListCompat
5	
6	
7	import androidx.appcompat.app.AppCompatActivityDelegate
8	import androidx.compose.foundation.layout.*
9	import androidx.compose.material3.*
10	import androidx.compose.runtime.*
11	import androidx.compose.ui.Alignment
12	import androidx.compose.ui.Modifier
13	import androidx.compose.ui.platform.LocalContext
14	import androidx.compose.ui.res.stringResource
15	import androidx.compose.ui.unit.dp
16	import androidx.lifecycle.viewmodel.compose.viewModel
17	import androidx.navigation.NavController
18	import com.example.listcompose.R
19	import com.example.listcompose.ViewModel.LanguageViewModel
20	import
21	com.example.listcompose.ViewModel.LanguageViewModelFactory
22	@Composable
23	fun LanguageScreen(navController: NavController) {
24	val context = LocalContext.current
25	val factory =
26	LanguageViewModelFactory(stringResource(R.string.languagepage))
27	val viewModel : LanguageViewModel = viewModel(factory =
28	factory)
29	LaunchedEffect(Unit) {
30	Toast.makeText(context, viewModel.pageInfo,
31	Toast.LENGTH_LONG).show()
31	}

```

32     val currentLocale =
AppCompatDelegate.getApplicationLocales().toLanguageTags()
33
34     var selectedOption by remember {
35         mutableStateOf(if (currentLocale.contains("in"))
"Bahasa" else "English")
36     }
37
38     Scaffold(
39         topBar = {
40             HeaderScreen(
41                 title = stringResource(R.string.app_title),
42                 buttonText =
stringResource(R.string.homebutton),
43                 icon = R.drawable.home,
44                 onClick = { navController.popBackStack() }
45             )
46         }
47     ) { innerPadding ->
48         Column(
49             modifier = Modifier
50                 .fillMaxSize()
51                 .padding(innerPadding)
52                 .padding(24.dp)
53         ) {
54             LanguageOption(
55                 label = stringResource(R.string.english),
56                 selected = (selectedOption == "English"),
57                 onClick = {
58                     selectedOption = "English"
59                     val appLocale: LocaleListCompat =
LocaleListCompat.forLanguageTags("en")
60
AppCompatDelegate.setApplicationLocales(appLocale)
61                 }
62             )
63
64             LanguageOption(
65                 label = stringResource(R.string.indonesia),
66                 selected = (selectedOption == "Bahasa"),
67                 onClick = {
68                     selectedOption = "Bahasa"
69                     val appLocale: LocaleListCompat =
LocaleListCompat.forLanguageTags("in")
70
AppCompatDelegate.setApplicationLocales(appLocale)
71                 }
72             )
73         }
74     }
75 }
76
77 @Composable
78 fun LanguageOption(label: String, selected: Boolean, onClick: ()

```

```

-> Unit) {
79     Row(
80         verticalAlignment = Alignment.CenterVertically,
81         modifier = Modifier
82             .fillMaxWidth()
83             .padding(vertical = 4.dp)
84     ) {
85         RadioButton(
86             selected = selected,
87             onClick = onClick
88         )
89         Text(
90             text = label,
91             style = MaterialTheme.typography.bodyLarge,
92             modifier = Modifier.padding(start = 8.dp)
93         )
94     }
95 }

```

Navigation

Nav_Graph.kt

Tabel 36. Source Code Soal 1 Nav_Graph.kt Compose

```

1  package com.example.listcompose.Navigation
2
3  import androidx.compose.runtime.Composable
4  import androidx.compose.ui.res.stringResource
5  import androidx.lifecycle.viewmodel.compose.viewModel
6  import androidx.navigation.NavDestination
7  import androidx.navigation.NavHostController
8  import androidx.navigation.NavType
9  import androidx.navigation.compose.NavHost
10 import androidx.navigation.compose.composable
11 import androidx.navigation.compose.rememberNavController
12 import androidx.navigation.navArgument
13 import com.example.listcompose.Data.DataSource
14 import com.example.listcompose.Layout.DetailScreen
15 import com.example.listcompose.Layout.HomeScreen
16 import com.example.listcompose.Layout.LanguageScreen
17 import com.example.listcompose.ViewModel.FilmViewModel
18 import com.example.listcompose.ViewModel.FilmViewModelFactory
19 import com.example.listcompose.R
20 import com.example.listcompose.ViewModel.DetailViewModel
21 import com.example.listcompose.ViewModel.DetailViewModelFactory
22
23 @Composable
24 fun Nav_Graph(
25     navController: NavHostController,
26     startDestination : String = "home"
27 ) {

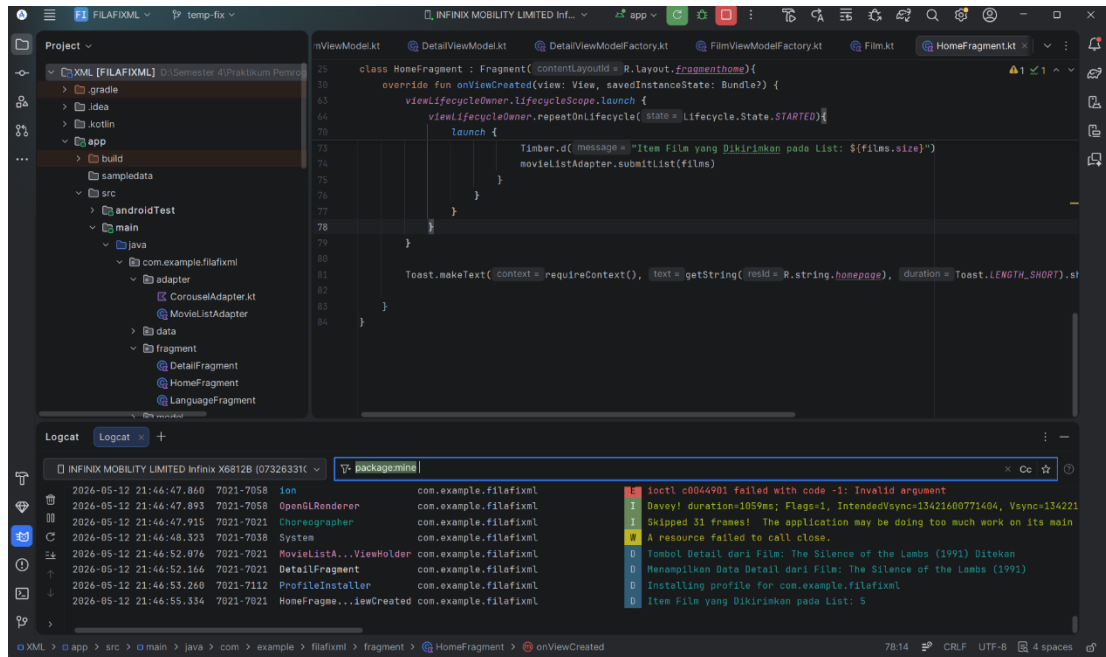
```

```

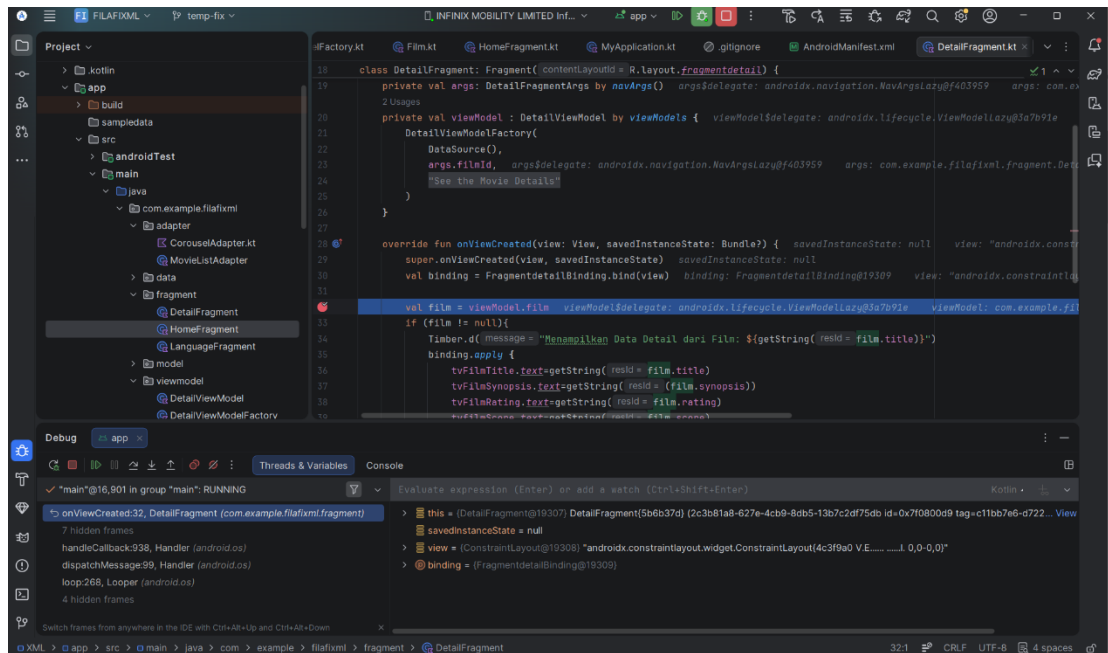
28     NavHost(
29         navController = navController,
30         startDestination = startDestination
31     ) {
32         composable("home") {
33             val factory = FilmViewModelFactory(DataSource(),
stringResource(R.string.homepage))
34             val viewModel : FilmViewModel = viewModel(factory =
factory)
35             HomeScreen(navController = navController, viewModel
= viewModel)
36         }
37
38         composable(
39             route = "detail/{filmId}",
40             arguments = listOf(navArgument("filmId") { type =
NavType.IntType })
41         ) { backStackEntry ->
42             val filmId =
backStackEntry.arguments?.getInt("filmId") ?: 0
43
44             val factory = DetailViewModelFactory(DataSource(),
filmId, stringResource(R.string.detailpage))
45             val viewModel : DetailViewModel = viewModel(factory
= factory)
46             DetailScreen(
47                 filmId = filmId,
48                 viewModel = viewModel,
49                 navController = navController
50             )
51         }
52         composable("language") {
53             LanguageScreen(navController = navController)
54         }
55     }
56 }

```

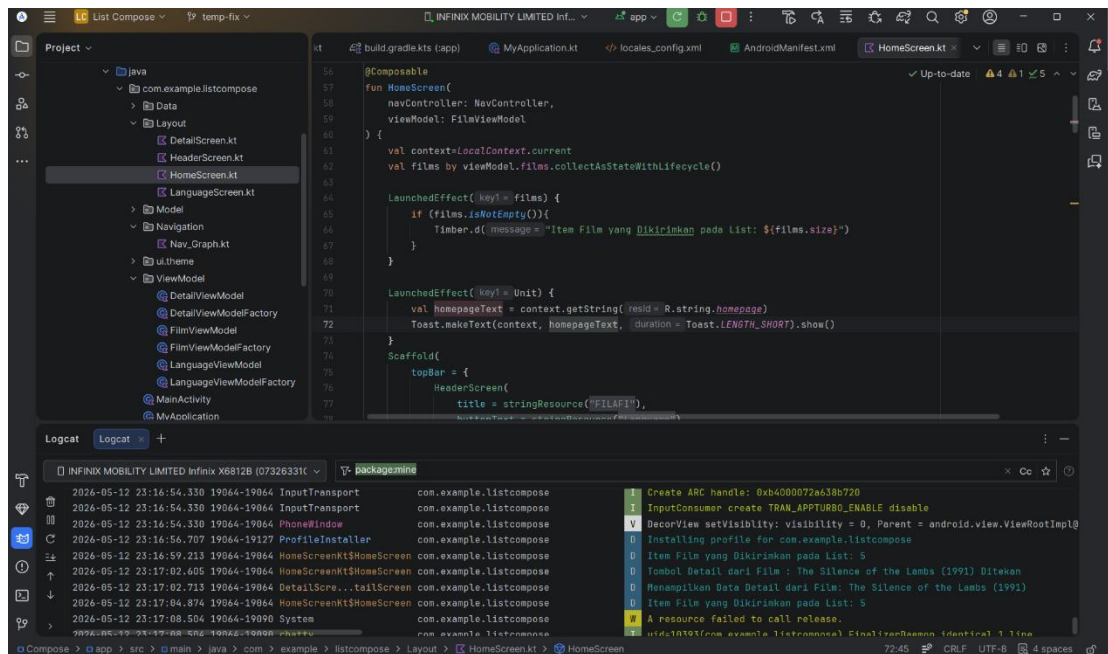
B. Output Program



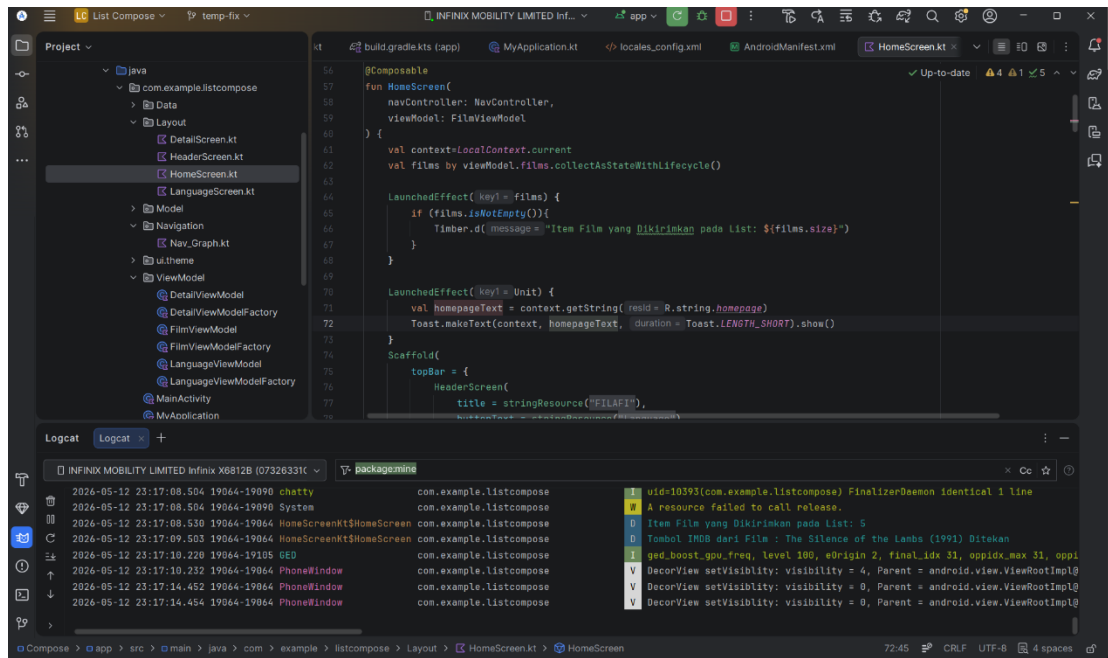
Gambar 1. Screenshot Hasil Logging Timber Soal 1 XML



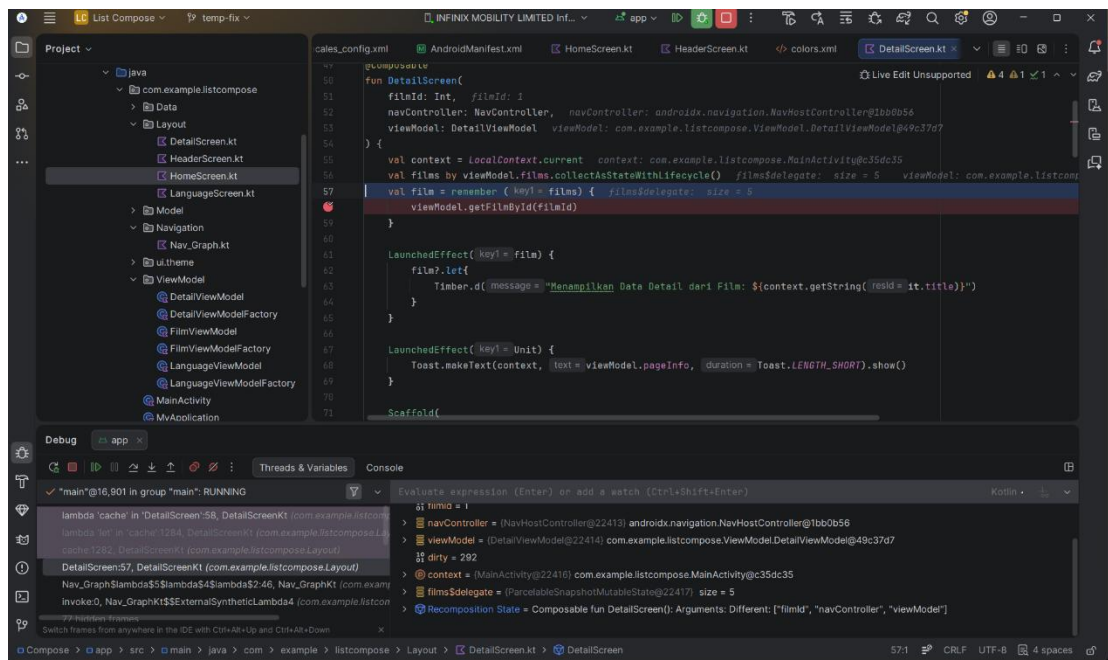
Gambar 2. Screenshot Hasil Debugging Timber Soal 1 XML



Gambar 3. Screenshot Hasil Logging Timber Soal 1 Compose



Gambar 4. Screenshot Hasil Logging Timber Soal 1 Compose



Gambar 5. Screenshot Hasil Debugging Timber Soal 1 Compose

C. Pembahasan

XML

MainActivity.kt XML

MainActivity berfungsi sebagai tempat dimulainya program lebih tepatnya pada onCreate yang akan menerima seluruh tampilan dari setContentView yang menerima seluruh data tampilan dari variabel binding yang diinisialisasi secara global. Selain itu pada MainActivity juga diatur untuk menjadi host dari proses navigasi dari keseluruhan aplikasi yang terdiri atas 3 fragment utama yakni home, detail, dan language.

Film.kt

Film.kt memiliki peran untuk menjadi wadah dari model data yang menyimpan data dari film yang akan ditampilkan pada list mulai dari id film yang akan digunakan dalam proses navigasi, hingga atribut dari film itu sendiri seperti synopsis, poster, hingga url IMDB dari yang akan digunakan untuk menjadi tujuan ketika tombol IMDB dari salah satu list film diklik. Atribut diatur agar nantinya akan mengembalikan nilai bertipe Int untuk menampung String resource yang disimpan pada Strings.xml, dan Drawable resource untuk atribut yang memerlukan gambar.

DataSource.kt

DataSource memiliki tugas untuk menjadi wadah diinisialisasinya sebuah function yang berfungsi untuk menyimpan data dari film secara list sehingga dapat menampung sekaligus semua nilai dari atributnya sekaligus seperti url dari atribut url, ataupun sumber foto dari atribut poster. Pada function inilah data lengkap dari kelima film disimpan untuk nantinya digunakan pada viewmodel.

FilmViewModelFactory.kt

FilmViewModelFactory yang baru dibuat pada modul ini berperan agar FilmViewModel bisa melakukan passing parameter, yang mana berdasarkan petunjuk untuk salah satu parameternya adalah String, di mana bisa digunakan sebagai pengatur nilai berupa isi pesan dari toast yang akan dimunculkan pada saat inisialisasi dari HomeFragment.

FilmViewModel.kt

FilmViewModel berfungsi sebagai wadah diinisialisasinya variabel yang menyimpan data yang bisa berubah dan sadar akan life cycle sehingga datanya akan tetap sama meskipun terjadi perubahan misalnya seperti perubahan ke mode landscape yang mana akan membuat efek reset kepada activity, di mana variabel yang diinisialisasi akan menyimpan seluruh data dari film, disinilah function dari DataSource digunakan, dan nantinya viewmodel lah yang akan berperan menjadi penyedia data dari fragment. Pada modul ini FilmViewModel diatur menggunakan MutableStateFlow yang salah satu keunggulannya dibandingkan LiveData seperti modul sebelumnya adalah mewajibkan adanya nilai pada saat inisialisasi sehingga lebih aman dari NullPointerException. Adapun karena adanya penambahan factory pada modul ini maka sekarang FilmViewModel memiliki parameter dataSource yang berisikan data dari class DataSource dan juga pageInfo yang bertipe String.

DetailViewModelFactory.kt

Sama dengan FilmViewModelFactory, class ini berfungsi agar pada DetailViewModel bisa melakukan passing parameter, yang mana salah satu parameternya adalah pageInfo bertipe String sebagaimana petunjuk dari modu, yang dapat digunakan untuk mengatur isi pesan dari toast saat DetailFragment di inisialisasi.

DetailViewModel.kt

DetailViewModel berperan untuk menginisialisasi variabel yang menyimpan logika bisnis, di mana ia digunakan untuk menyimpan seluruh data film dari DataSource dan melakukan pengecekan apakah film yang dimaksud oleh pengguna saat menekan tombol detail ada pada isi data yang disimpan oleh variabel.

LanguageViewModelFactory.kt

Sama dengan dua ViewModelFactory sebelumnya, class ini juga berfungsi agar pada LanguageViewModel bisa melakukan passing parameter yakni pageInfo bertipe String yang nantinya akan digunakan untuk mengatur isi pesan toast saat LanguageFragment diinisialisasi.

LanguageViewModel.kt

ViewModel ini hanya difungsikan agar nantinya pada LanguageFragment bisa menerima parameter pageInfo untuk mengatur isi pesan dari toast yang akan dijalankan pada saat inisialisasi fragment.

CarouselAdapter.kt

CarouselAdapter berfungsi sebagai penerima data yang sudah ditampung oleh viewmodel sebelumnya, di mana disini, pertama-tama dilakukan pengecekan apakah data film masih sama, kemudian onCreateViewHolder diatur untuk menginisialisasi variabel yang diberikan informasi tampilan layout, dan akan diteruskan menuju inner class. Selanjutnya layout akan diatur untuk menampilkan data oleh function yang diinisialisasi pada inner class serta carousel diatur mengirimkan id ketika di klik untuk nantinya akan digunakan pada fragment, barulah function akan digunakan oleh onBindViewHolder yang menerima informasi posisi pada layout sehingga data akan ditempelkan pada layout secara tepat.

MovieListAdapter.kt

MovieListAdapter bekerja dengan logika yang sama seperti CarouselAdapter di mana akan menerima data dari viewmodel, dicek apakah datanya masih sama, menerima informasi dari layout dan diteruskan ke inner class, pada inner class juga dibuat function untuk menempelkan data dan mengatur agar tombol dan bagian lainnya dari list saat diklik akan mengirimkan id film, lalu ditempelkan oleh onBindViewHolder yang menerima informasi posisi dari layout sehingga data dapat ditempelkan secara tepat sasaran, hanya saja disini yang diatur adalah tampilan dari layout list film.

HomeFragment.kt

HomeFragment berperan mengatur logika navigasi menggunakan komponen dari nav_graph dengan mengandalkan id yang sebelumnya diatur oleh adapter untuk dikirimkan ketika dilakukan klik baik pada carousel dan list film, selanjutnya mengatur agar recyclerview menerima data dari layout, dan mengatur logika navigasi dari

tombol pada header, barulah diatur agar carousel serta list menyesuaikan jumlahnya dengan jumlah film yang ada pada data. Pada Modul ini terdapat perubahan pada HomeFragment seperti inisialisasi variabel viewModel yang menggunakan delegasi dengan tujuan agar ViewModel tidak ikut hancur saat HomeFragment hancur misalnya saat perubahan mode handphone dari vertikal ke horizontal, kini menggunakan Factory untuk menerima parameter yakni DataSource() dan string resource yang berfungsi menjadi isi pesan dari toast. Namun karena permasalahan di mana bahasa dari pesan toast tidak berganti meski pengaturan bahasa telah di Ganti, maka yang awalnya pada toast menggunakan nilai dari viewModel harus diganti menjadi langsung memanggil string resourcenya. Juga terjadi perubahan syntax pada pengiriman data dan inisialisasi dari list highlight dan film, serta penggunaan timber pada pengaturan logika tombol IMDB dan Detail.

DetailFragment.kt

DetailFragment berperan menerima argumen dari proses navigasi berupa id dari film, yang mana dia akan memeriksa apakah film dengan id tersebut eksis atau tidak, disini juga menerima informasi dari layout fragmentdetail yang akan diatur untuk menampilkan data dari viewModel menyesuaikan dengan id dari film, misalnya jika film dengan id 1 yakni The Silence of the Lambs diterima maka layout diatur untuk menerima data dari viewModel berupa atribut dari film tersebut. Sama seperti HomeFragment inisialisasi variabel viewModel juga menggunakan factory untuk menerima parameter berupa DataSource(), filmId, dan pageInfo. Juga terdapat penggunaan timber pada bagian di mana pengiriman data menuju fragment detail mulai dilakukan

LanguageFragment.kt

LanguageFragment berfungsi untuk mengatur logika ketika pengguna memilih salah satu pilihan dari radio button, yang mana akan dijalankan fungsi changeLanguage yang akan mengambil informasi bahasa dari locale berdasarkan kode bahasa yakni untuk Indonesia adalah “in”, dan untuk Inggris adalah “en”. LanguageFragment juga mengalami perubahan yang sama di mana parameter viewModel diinisialisasi dengan

Factory sehingga bisa menerima parameter yakni pageInfo untuk mengatur isi dari pesan toast.

Activity_Main.xml

Activity_Main berperan sebagai kerangka dasar atau fondasi tampilan dari keseluruhan aplikasi yang membungkus seluruh elemen visual menggunakan ConstraintLayout agar tata letaknya dapat menyesuaikan dimensi layar secara fleksibel. Layout ini memiliki peran penting untuk menampung dua komponen utama, yakni komponen Toolbar yang diletakkan menempel pada bagian atas layar untuk berperan sebagai header atau bar aplikasi, serta komponen krusial berupa FragmentContainerView yang diatur sebagai NavHostFragment. Komponen FragmentContainerView inilah yang ditugaskan untuk menjadi wadah utama atau host tempat terjadinya proses penukaran antarmuka, di mana ia akan membaca alur dari nav_graph sehingga tampilan dari ketiga fragment utama seperti home, detail, dan language dapat dimunculkan dan diganti-ganti secara dinamis di dalam area wadah tersebut seiring dengan interaksi navigasi dari pengguna.

fragmenthome.xml

fragmenthome berfungsi sebagai penyusun antarmuka halaman utama atau beranda dari aplikasi yang membungkus keseluruhan elemen visual menggunakan ConstraintLayout sebagai fondasi dasarnya agar tata letak tetap responsif pada berbagai ukuran layar. Layout ini memiliki peran penting untuk membagi layar menjadi dua bagian utama yang statis dan dinamis, dimulai dari komponen MaterialCardView pada bagian paling atas yang dirancang sebagai header statis untuk menampung TextView judul aplikasi bersanding dengan sebuah ImageButton untuk fitur pergantian bahasa. Tepat di bawah area header tersebut, ditempatkan komponen krusial berupa NestedScrollView yang berfungsi memastikan seluruh konten di bawahnya dapat dilakukan scroll secara vertikal oleh pengguna. Di dalam wadah gulir inilah ditata susunan konten utamanya, yang terdiri atas sebuah MaterialCardView berisi komponen RecyclerView pertama yang diatur horizontal untuk menampilkan deretan carousel

sorotan film, dilanjutkan dengan sebuah TextView sebagai pembatas atau subjudul, dan diakhiri dengan komponen RecyclerView kedua yang ditugaskan untuk merender daftar keseluruhan film secara vertikal sehingga pengguna dapat menjelajahi katalog film sesuai data yang disediakan oleh viewmodel.

Fragmentdetail.xml

fragmentdetail berfungsi sebagai penyusun antarmuka halaman detail yang bertugas menampilkan informasi lengkap secara spesifik dari sebuah film, dibungkus menggunakan ConstraintLayout sebagai fondasi dasarnya agar tata letaknya fleksibel. Layout ini mengandalkan dua area utama, yakni komponen include pada bagian atas yang menyematkan layout header statis untuk fitur navigasi kembali ke halaman home, lalu NestedScrollView tepat di bawahnya yang memastikan seluruh konten detail dapat dilakukan scroll secara vertikal oleh pengguna. Di dalam wadah komponen inilah seluruh komponen informatifnya diatur, yang diawali dengan dua MaterialCardView untuk menampilkan gambar yakni poster atau gambar dari film, dilanjutkan dengan rentetan TextView untuk menampilkan data berupa judul, sinopsis, hingga angka rating, dan ditutup oleh sebuah MaterialCardView di area paling bawah yang dirancang khusus sebagai wadah untuk menampilkan teks ulasan atau komentar mengenai film tersebut.

fragmentlanguage.xml

fragmentlanguage berperan sebagai kerangka antarmuka untuk halaman pengaturan bahasa yang dibungkus menggunakan ConstraintLayout sebagai fondasi dasarnya. Layout ini dirancang jauh lebih sederhana dan langsung pada tujuannya, mengandalkan komponen include pada bagian atas untuk menyematkan layout header statis yang memfasilitasi fitur navigasi kembali. Tepat di bawah header tersebut, diletakkan komponen krusial berupa RadioGroup yang bertugas sebagai wadah untuk menampung dua pilihan bahasa dalam bentuk RadioButton dengan opsi bahasa Inggris

dan bahasa Indonesia yang diatur secara vertikal sehingga pengguna hanya dapat memilih salah satu pengaturan bahasa secara eksklusif.

Headerback.xml

headerback memiliki peran untuk mengatur tampilan dari header statis yang akan ditampilkan pada halaman detail dan pemilihan bahasa, di mana tampilannya diatur menggunakan ConstraintLayout, CardView, Textview, dan ButtonImage.

Item_film.xml

item_film berperan untuk mengatur tampilan yang nantinya akan dikirimkan pada recylerview pada halaman home menggunakan komponen CardView, Button, TextView, dan ImageView, serta pemosisian diatur menggunakan ConstraintLayout.

Item_highlight.xml

Item_highlight berfungsi mengatur tampilan yang akan dikirimkan pada recylerview pada halaman home lebih tepatnya sebagai carousel, yang tampilannya diatur menggunakan ImageView dan CardView, dan menggunakan ConstraintLayout dalam penyusunannya.

Compose

MainActivity.kt

MainActivity berfungsi sebagai tempat dimulainya program lebih tepatnya pada onCreate yang akan menerima dan mengatur seluruh tampilan dari setContent berbasis fungsi composable. Di dalam setContent inilah tema aplikasi dari ListComposeTheme diterapkan, serta diinisialisasi variabel navController untuk menangani proses navigasi dan viewModel sebagai wadah penyedia data. Selain itu pada MainActivity juga diatur untuk menjadi host utama dari keseluruhan aplikasi dengan menempatkan komponen Nav_Graph di atas sebuah Surface yang mengatur warna latar dari aplikasi, di mana Nav_Graph tersebut akan menerima navController dan pada modul ini ViewModel digantikan dengan StartDestination yang diatur menjadi home, agar nantinya dapat diteruskan dan digunakan pada seluruh proses navigasi halaman aplikasi.

Film.kt

Film.kt memiliki peran untuk menjadi wadah dari model data yang menyimpan data dari film yang akan ditampilkan pada list mulai dari id film yang akan digunakan dalam proses navigasi, hingga atribut dari film itu sendiri seperti synopsis, poster, hingga url IMDB dari yang akan digunakan untuk menjadi tujuan ketika tombol IMDB dari salah satu list film diklik. Atribut diatur agar nantinya akan mengembalikan nilai bertipe Int untuk menampung String resource yang disimpan pada Strings.xml, dan Drawable resource untuk atribut yang memerlukan gambar.

DataSource.kt

DataSource memiliki tugas untuk menjadi wadah diinisialisasinya sebuah function yang berfungsi untuk menyimpan data dari film secara list sehingga dapat menampung sekaligus semua nilai dari atributnya sekaligus seperti url dari atribut url, ataupun sumber foto dari atribut poster. Pada function inilah data lengkap dari kelima film disimpan untuk nantinya digunakan pada viewmodel.

FilmViewModelFactory.kt

FilmViewModelFactory yang baru dibuat pada modul ini berperan agar FilmViewModel bisa melakukan passing parameter, yang mana berdasarkan petunjuk untuk salah satu parameternya adalah String, di mana bisa digunakan sebagai pengatur nilai berupa isi pesan dari toast yang akan dimunculkan pada saat inisialisasi dari HomeScreen.

FilmViewModel.kt

FilmViewModel.kt berperan untuk menjadi wadah yang menampung data dari model dan DataSource, di mana diinisialisasi variabel untuk menampung data dari DataSource, dan diinisialisasi variabel untuk menampung semua data yang disimpan pada function listFilm pada DataSource, serta variabel yang tipe datanya adalah StateFlow yang berisikan List dari model Film di mana datanya dipastikan selalu sama

dengan variabel yang menampung data dari function listFilm. Juga dilakukan pembuatan function yang menggunakan nilai dari variabel _films yakni variabel penerima data dari function listFilm untuk melakukan pencaharian apakah id yang menjadi input sama dengan yang disimpan pada data. Adapun perbedaan dengan modul sebelumnya adalah penggunaan StateFlow serta kini ViewModel bisa menerima parameter dari FilmViewModelFactory.

DetailViewModelFactory.kt

Sama dengan FilmViewModelFactory, class ini berfungsi agar pada DetailViewModel bisa melakukan passing parameter, yang mana salah satu parameternya adalah pageInfo bertipe String sebagaimana petunjuk dari modul, yang dapat digunakan untuk mengatur isi pesan dari toast saat DetailFragment di inialisasi.

DetailViewModel.kt

Isi dari DetailViewModel sama dengan isi dari FilmViewModel pada modul sebelumnya dengan logika yang sama serta function yang sama dengan fungsi untuk menyimpan nilai dari DataSource() dan logika untuk memeriksa apakah film yang dimaksud pengguna saat menekan tombol detail sama dengan data film yang ada pada list, hanya saja sesuai dengan instruksi digunakan stateflow, serta ViewModel menerima parameter dari factory.

LanguageViewModelFactory.kt

Sama dengan dua ViewModelFactory sebelumnya, class ini juga berfungsi agar pada LanguageViewModel bisa melakukan passing parameter yakni pageInfo bertipe String yang nantinya akan digunakan untuk mengatur isi pesan toast saat LanguageScreen diinisialisasi.

LanguageViewModel.kt

LanguageViewModel hanya berperan untuk menerima parameter pageInfo yang nantinya akan digunakan pada LanguageScreen untuk mengatur isi pesan toast saat screen dijalankan.

HeaderScreen.kt

HeaderScreen berfungsi untuk mengatur tampilan header statis yang nantinya dapat dipanggil pada halaman home, language. dan detail, di mana function HeaderScreen diatur agar menerima parameter berupa judul, teks untuk tombol, icon yang diatur agar menerima resource drawable, serta logika jika tombol dilakukan klik. Pengaturan tampilan diatur menggunakan Surface yang diberi pengaturan warna dan dibuat memaksimalkan lebar yang tersedia pada layar. Dengan komponennya disusun menggunakan Row, Text, dan Icon yang dibuat agar bisa diklik.

HomeScreen.kt

HomeScreen berperan untuk mengatur tampilan dari halaman home, di mana pertama-tama dilakukan pemanggilan kepada HeaderScreen dengan parameternya diisi menggunakan StringResource, DrawableResource, dan logika navigasi dari NavController. Lalu dilakukan pengaturan tampilan dengan membuat kerangka layout yang diatur agar nantinya isinya menggunakan isi dari dua function yang diinisialisasi setelahnya yakni HighlightCard, dan MovieItem di mana keduanya juga berisikan pengaturan tampilan yang diatur agar menampilkan data bermodalkan data dari viewmodel. Perubahan pada modul ini terdapat pada inisialisasi variabel yang menyimpan konteks lokal untuk nanti dijadikan konteks pada toast. Kemudian ada inisialisasi variabel films yang nantinya akan digunakan pada logika pengiriman data ke komponen dari homescreen, di mana digunakan delegasi dan juga collectAsStateWithLifecycle yang memiliki keunggulan di mana daya baterai handphone dapat menjadi lebih hemat karena mencegah pengiriman data saat aplikasi sedang tidak dijalankan atau tidak dilihat oleh pengguna. Selain itu juga ditambahkan Launch untuk keperluan logging menggunakan timber di mana bertujuan untuk mengetahui jumlah data yang dikirimkan pada list. Masalah yang terjadi pada versi compose di mana pesan toast tidak bisa ditranslasi otomatis saat mengganti bahasa membuat akhirnya saya menginisialisasi variabel untuk memanggil langsung string resource yang diperlukan.

DetailScreen.kt

DetailScreen berfungsi untuk mengatur tampilan dari halaman detail, di mana pertama-tama dilakukan pencarian data film berdasarkan id yang diterima untuk dicocokkan menggunakan function dari viewmodel. Selanjutnya dilakukan pemanggilan kepada HeaderScreen dengan parameternya diisi menggunakan StringResource, DrawableResource, dan logika navigasi kembali ke halaman home dari navController. Lalu dilakukan pengaturan tampilan dengan membuat kerangka layout berupa kondisi pengecekan data, di mana jika data film tersedia maka layout diatur agar isinya menampilkan komponen gambar besar, teks judul dan sinopsis, gambar poster, teks rating, hingga Card ulasan berdasarkan data spesifik dari viewmodel yang disusun berurutan pada kolom yang bisa digulir, dan jika tidak tersedia maka diatur agar isinya menampilkan pesan error. Perubahan pada modul ini terdapat pada inisialisasi variabel yang menyimpan konteks lokal untuk nanti dijadikan konteks pada toast. Kemudian ada inisialisasi variabel films yang nantinya akan digunakan pada logika pengiriman data ke komponen dari detailscreen, di mana digunakan delegasi dan juga collectAsStateWithLifecycle yang memiliki keunggulan di mana daya baterai handphone dapat menjadi lebih hemat karena mencegah pengiriman data saat aplikasi sedang tidak dijalankan atau tidak dilihat oleh pengguna. Selain itu juga ditambahkan Launch untuk keperluan logging menggunakan timber di mana bertujuan untuk mengetahui jumlah data yang dikirimkan pada list.

LanguageScreen.kt

LanguageScreen berperan untuk mengatur tampilan dari halaman pengaturan bahasa, di mana pertama-tama dilakukan inisialisasi untuk mendapatkan informasi bahasa yang sedang digunakan oleh aplikasi dari sistem, dan menyimpannya ke dalam variabel state agar status pilihan bisa diperbarui secara otomatis. Selanjutnya dilakukan pemanggilan kepada HeaderScreen dengan parameternya diisi menggunakan StringResource, DrawableResource, dan logika navigasi kembali ke halaman home dari navController. Lalu dilakukan pengaturan tampilan dengan membuat kerangka layout berupa kolom yang diatur agar isinya menampilkan dua komponen dari function LanguageOption. LanguageOption di sini berfungsi murni untuk menampilkan bentuk

visual pilihan bahasa menggunakan baris (Row) sehingga RadioButton dan Text berjejer rapi, di mana ia bekerja dengan menerima argumen dari LanguageScreen berupa label teks, informasi status pilihan, dan logika klik yang diteruskan ke RadioButton untuk dieksekusi secara tepat sasaran yang mana akan langsung memperbarui state dan menerapkan perubahan bahasa aplikasi secara menyeluruh menggunakan tag bahasa, yakni "en" untuk Inggris dan "in" untuk Indonesia. Adapun untuk modul ini pada class ini terdapat perubahan pada inialisasi variabel viewModel di mana menggunakan factory untuk menerima parameter, cara ini berbeda dengan inialisasi pada home dan detail screen dikarenakan tidak seperti keduanya yang parameternya adalah DataSource() serta filmId yang membuat inialisasi factory keduanya lebih baik dilakukan pada nav_graph, languagescreen hanya memerlukan pageInfo sehingga dia bisa lebih fleksibel untuk menginisialisasi factory langsung pada screen.

Nav_Graph.kt

Nav_Graph berfungsi sebagai pusat kendali untuk mengatur seluruh alur navigasi dari aplikasi, di mana komponen ini menerima argumen berupa navController untuk menangani proses perpindahan halaman dan viewModel sebagai jalur distribusi data utama. Di sini, dilakukan pengaturan kerangka navigasi menggunakan komponen NavHost yang diatur agar menetapkan "home" sebagai halaman pertama yang dimuat saat aplikasi mulai dijalankan. Selanjutnya diatur konfigurasi dari masing-masing tujuan atau rute, mulai dari rute "home" yang diatur agar menampilkan HomeScreen bermodalkan navController dan viewModel, rute "detail" yang diatur dengan logika untuk menerima argumen spesifik yakni id film bertipe Integer dari jalur navigasinya untuk diteruskan menuju DetailScreen bersama parameter lainnya, hingga rute "language" yang diatur agar memanggil LanguageScreen bermodalkan navController saja untuk kebutuhan navigasinya yakni kembali ke halaman home. Adapun pada modul ini dilakukan perubahan di mana dilakukan inialisasi factory pada home dan detail dikarenakan keduanya memerlukan parameter yang berasal dari navigasi seperti

filmId, dan DataSource() yang datanya berasal dari eksternal sehingga lebih baik inisialisasi dilakukan pada navigation agar class UI menjadi lebih clean.

Untuk Timber yang sesuai intruksi dilakukan untuk mencatat pada saat tombol detail dan IMDB ditekan, data dikirimkan pada list, dan keadaan list saat data dikirimkan pada detail, maka Timber diletakkan pada MovieListAdapter, FilmViewModel, HomeFragment, dan DetailFragment pada XML, sementara pada Compose diletakkan pada HomeScreen dan DetailScreen. Adapun loggingnya bisa dilihat pada logcat dan akan terlihat jumlah data yang dikirimkan pada list, tombol IMDB dan Detail dari film apa yang dilakukan klik, serta data detail dari film apa yang diklik,

Adapun debugging dilakukan pada titik di mana data dari viewModel yang berisikan data film dikirimkan, sehingga jika dilakukan step into maka akan beralih ke sumber data yakni saat inisialisasi variabel viewModel, lalu step out akan mengembalikan ke baris semula, dan step over akan menjalankan baris berikutnya di mana data yang dikirimkan akan terlihat.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya!

A. Pembahasan

Dari apa yang saya pahami, Application class itu adalah class yang menjadi titik masuk pertama dari aplikasi yang kita buat, bahkan lebih dulu diinstansiasi sebelum activity, service, dan broadcast receiver, sehingga karena itulah seperti pada modul ini Application class itu salah satunya dapat difungsikan untuk menginisialisasi library dari luar seperti pada kasus modul ini adalah Timber, lalu didaftarkan pada AndroidManifest.xml. Hal tersebut dikarenakan ia menjadi titik masuk pertama kali seperti yang dijelaskan sebelumnya sehingga akan menjamin library siap untuk digunakan. Selain itu Application juga akan terus hidup selama aplikasi berjalan sehingga hanya perlu 1 kali inisialisasi saat awal aplikasi dijalankan, dan tidak perlu dilakukan setiap perpindahan activity atau fragment.

TAUTAN GIT

<https://github.com/HamkaArifani/Praktikum-Pemrograman-Mobile>